

Physics-informed neural networks on the van der Pol oscillator: a measured comparison of the continuous-time and discrete-time formulations

George William Scriven*

Maastricht University, Maastricht, The Netherlands

Hasselt University, Hasselt, Belgium

Nikhef, Amsterdam, The Netherlands

August 19, 2026

Abstract

We rebuild the two formulations of physics-informed neural networks (PINNs) introduced by Raissi, Perdikaris and Karniadakis on the van der Pol oscillator and measure both against a fixed-step sixth-order Runge–Kutta reference trusted to 10^{-13} , at least six orders of magnitude below any trained network in this study. The continuous-time formulation, transcribed faithfully, matches or betters the accuracy reported in the original work on horizons up to one period of the limit cycle (3.8×10^{-5} to 6.6×10^{-4} relative L_2 error). Beyond one period it fails structurally: every seed collapses onto a spurious solution of the differential equation while the training loss converges, so the failure is silent. Across 733 training runs, no resource changes where this happens. Causal weighting of the residuals, the strongest loss-level modification available, moves the failure from one period out to two at a cost that grows faster than linearly. A four-fold extension of the optimisation budget repairs none of the seven runs it was applied to. A 600-run sweep over 25 architectures finds no size that repairs a failing horizon, and added depth is actively harmful. Varying the collocation density 64-fold has no effect on the plain formulation at the failing horizons, and targeted re-placement of the collocation points changes nothing. What does order the outcomes is the starting state: at two periods the causally weighted formulation reaches better than 1% error for 12.7% of the architecture and start combinations tested, with the successes concentrated on the outer starts, and at four periods for none. The discrete-time formulation, generalised so that the network input is the starting state itself, learns a single reusable implicit Runge–Kutta step of size $\Delta t = 0.8$ from the equations alone. Swept over the number of stages against the exactly solved scheme, it is scheme-limited at $q = 2$ and network-limited from $q = 4$, with an endpoint error floor of 3.6×10^{-3} that a 135-run architecture sweep shows to be an optimisation floor rather than a capacity one (rank correlation between training loss and held-out one-step error +0.974). Long-horizon quality is decided by the training seed, is measurable on a validation chain, and transfers. A simple selection protocol (train ten seeds, chain each to the target horizon on validation starts, keep the best) yields a single network of 4 layers and 32 units that carries 108 held-out starting states through six periods at 2.0×10^{-3} relative L_2 error, a horizon at which even the causally weighted continuous-time formulation has an error above one, and that stays flat to thirty periods on fresh starts. Chaining this map beats the source work’s single-large-step mode by a factor of 10 at the largest feasible step. Finally, a matched data-supervised twin quantifies the accuracy cost of training from the equations alone: the equation-trained network lands within a factor 2.1 of perfect-label training at zero reference-data cost.

*george.w.scriven@gmail.com

Contents

1	Introduction	2
2	Test problem, reference solution, and metrics	4
2.1	The van der Pol oscillator	4
2.2	The reference solution	5
2.3	Error measures	5
3	The continuous-time formulation	6
3.1	Method	6
3.2	Collocation budget	7
3.3	Protocol	7
3.4	Results: faithful reproduction	8
3.5	Causal weighting and its limits	11
3.5.1	Method	11
3.5.2	Results	12
3.5.3	The budget-extension test	14
3.6	Network size and the starting state	16
3.7	Collocation density	18
3.8	Collocation placement	20
3.9	What the continuous-time route establishes	21
4	The discrete-time formulation	21
4.1	Implicit Runge–Kutta background	21
4.2	The loss	23
4.3	The state as the input: a reusable one-step map	23
4.4	Protocol and guardrails	24
4.5	Results: the error floor	24
4.6	The capped measure, and the error as a distribution	25
4.7	Network size, the seed, and selection	26
4.8	Whole trajectories: chaining versus one large step	27
4.9	Training from labels instead: a matched comparison	30
5	Discussion	32
6	Future work	34
7	Conclusion	34

1 Introduction

A physics-informed neural network (PINN) is a neural network trained to satisfy a differential equation: the training loss contains the equation itself, evaluated on the network’s own output by automatic differentiation, in place of (or in addition to) measured data. The canonical statement of the idea is the work of Raissi, Perdikaris and Karniadakis [1], referred to throughout this paper as *the source work*. It contains two distinct constructions:

1. a **continuous-time** formulation, in which the network *is* the solution (time goes in, and space too for a partial differential equation; the solution value comes out), with the equation enforced at sampled points of the domain; and

2. a **discrete-time** formulation, in which the network predicts the internal stage states of one implicit Runge–Kutta step and the algebra of the step itself supplies the supervision.

In this paper I rebuild both constructions from scratch on a single, completely controlled test problem, the van der Pol oscillator [2], a two-dimensional nonlinear system with an attracting limit cycle, and measure them under identical optimiser discipline against a reference solution whose own error is negligible. The oscillator is small enough to measure exhaustively and hard enough to defeat naive fits. My wider motivation is the construction of learned surrogates for ordinary differential equations arising in charged-particle track reconstruction [3], for which a controlled proving ground was required. The study asks where each formulation stops working, by what mechanism, and what the strongest available modification costs, rather than simply whether it can work. The evidence base is 733 training runs on the continuous-time side and 241 on the discrete-time side.

The contributions are:

1. a faithful reproduction of the continuous-time formulation that matches the source work’s reported accuracy up to one period of the limit cycle and fails beyond it by collapsing onto spurious solutions of the equation with a converged training loss. The problem is well-posed with a unique solution and the collocation budget is trivially affordable, so this is a measured counterexample to the source work’s stated expectation that a sufficiently expressive network with sufficiently many collocation points achieves good accuracy on a well-posed problem (Sections 3 and 3.4);
2. a demonstration that the failure is structural, by measuring that no resource changes the outcome: a four-fold optimisation budget repairs none of the seven runs that ended with the step budget spent (Section 3.5); a 600-run sweep over 25 architectures and six starting states finds no size that repairs a failing horizon, with added depth harmful (Section 3.6); the collocation density, varied over a 64-fold range, moves the plain formulation’s failing-horizon medians by at most a factor 1.15 (Section 3.7); and targeted re-placement of the collocation points changes nothing (Section 3.8);
3. an identification of what does order the outcomes: the starting state. Success at two periods occurs in 19 of the 150 causally weighted runs (12.7%) at the 1% error threshold and depends on where the start sits; at four periods none of the 300 runs of either variant succeeds (Section 3.6);
4. a measurement of causal weighting [4], the strongest loss-level modification in the literature: it moves the failure from one period out to two at a cost that grows faster than linearly, makes failures detectable in training telemetry, and carries a density floor (approximately 20 collocation points per unit time here) below which its weight-based completion condition is met on wrong solutions (Sections 3.5 and 3.7);
5. a generalisation of the discrete-time formulation in which the network input is the starting state itself, yielding a single reusable map that advances any state by a fixed time step, trained from the equations alone, following the extension of Fernández Corral et al. [5] (Section 4);
6. an attribution of the discrete-time map’s error: sweeping the number of implicit Runge–Kutta stages against the exactly solved scheme shows it scheme-limited at $q = 2$ and network-limited from $q = 4$ with a floor of 3.6×10^{-3} (Section 4.5). A 135-run architecture sweep then shows the floor is an optimisation floor rather than a capacity one: training loss and held-out one-step error rank-correlate at +0.974, so networks that fail also fail to train (Section 4.7);

7. a finding that long-horizon quality under chaining is decided by the training seed rather than the architecture, together with a selection protocol that exploits it: chained error on a validation set predicts held-out six-period error at rank correlation $+0.98$, and selecting the best of ten seeds yields a single network that holds 2.0×10^{-3} relative error to six periods and stays flat to thirty (Section 4.7);
8. a head-to-head comparison over identical horizons: the selected map, chained without retraining, completes six periods at 2.0×10^{-3} relative error where the causally weighted continuous-time formulation has an error above one, and beats the alternative reading of the source work in which one network learns a single large step covering the whole horizon by a factor 10 at the largest feasible step, which is bounded by nonlinear solvability rather than stability (Section 4.8); and
9. a measurement of what the physics loss costs in accuracy: against a twin trained on perfect reference labels (799 integrator steps per training state), the equation-trained network lands within a factor 2.1 at zero reference-data cost, and the gap is an optimisation gap (Section 4.9).

The paper is organised as follows. Section 2 fixes the test problem, the reference solution and the error measures, which everything later is scored against. Section 3 is the complete continuous-time route: the faithful reproduction and its failure mechanism, causal weighting with its budget-extension test, and the three studies that measure the remaining resources. Section 4 is the complete discrete-time route: the implicit Runge–Kutta background, the reusable one-step map, the error attribution, the seed and selection analysis, whole trajectories by chaining against one large step, and the data-trained comparison. Section 5 relates the measurements to the claims of the source work and of the causal-weighting literature; Section 6 lists the studies I would run next; and Section 7 concludes.

2 Test problem, reference solution, and metrics

2.1 The van der Pol oscillator

I use the van der Pol oscillator in first-order form, with state $y = (y_1, y_2)$ (position and velocity) and damping parameter μ fixed at 1 throughout:

$$\frac{dy_1}{dt} = y_2, \quad \frac{dy_2}{dt} = \mu(1 - y_1^2)y_2 - y_1. \quad (1)$$

Reading the second equation term by term: for $|y_1| < 1$ the “damping” term $\mu(1 - y_1^2)y_2$ has the same sign as the velocity and pumps energy in; for $|y_1| > 1$ it opposes the velocity and takes energy out. The result is the system’s defining feature, a limit cycle: a closed loop in the (y_1, y_2) plane onto which every trajectory except the equilibrium at the origin spirals, and around which it then circulates. One period is 6.663287 time units, and the loop’s extent is $|y_1| \leq 2.009$, $|y_2| \leq 2.678$ (both measured from the reference solution below; the period agrees with the accepted value). The equilibrium at the origin is repelling: the Jacobian of f there has eigenvalues $0.5 \pm 0.866i$, whose positive real part means nearby solutions grow like $e^{0.5t}$ while rotating, spiralling outward. Yet the origin satisfies Eq. (1) exactly. Both facts drive the failure analysis of Section 3.4. “Solving” the problem means producing the trajectory from a given starting state over a horizon T ; the canonical start throughout is $(2, 0)$, a point on the loop (Figure 1).

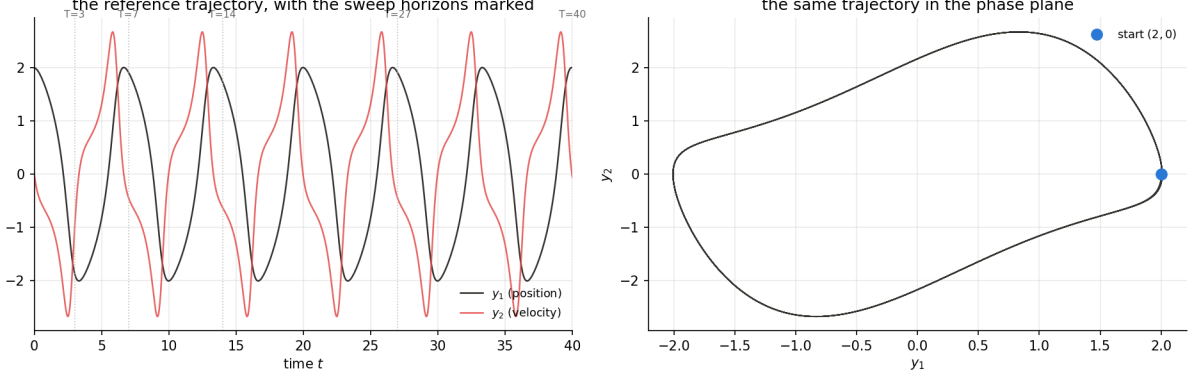


Figure 1: The test problem. Left: the reference trajectory from $(2,0)$ over six periods, with the five training horizons of the continuous-time experiments marked. Right: the same trajectory in the phase plane, the closed loop to be learned.

2.2 The reference solution

All errors are measured against a fixed-step, seven-stage, sixth-order explicit Runge–Kutta integration of Eq. (1) at step 10^{-3} in double precision, verified independently of any network by four instruments, each measured rather than asserted:

1. the convergence order, measured as the slope of the global error against the step size on logarithmic axes over three problems with known closed-form solutions (an order- p method appears as slope p); the measured slopes are 6.06, 6.27 and 5.75 against the theoretical 6;
2. step-halving self-consistency: integrating at h and $h/2$ and comparing, the disagreement reaches a rounding floor of 6.4×10^{-14} below $h \approx 5 \times 10^{-3}$, so at the working step 10^{-3} the truncation error is beneath double-precision rounding;
3. the period of the limit cycle, 6.663287 measured against the accepted value 6.6633; and
4. the closure of the limit cycle onto itself after one full period: 5.6×10^{-14} .

The reference is trustworthy to approximately 10^{-13} , at least six orders of magnitude below anything a trained network reaches in this study, so the yardstick never contaminates the measurement. The reference is used only for scoring, never for training, with one deliberate exception: the data-trained comparison of Section 4.9, whose entire point is to consume it. Where a network’s evaluation times fall between stored reference samples (times inside a single step, needed in Section 4), the reference integrator is re-run to stop exactly at each required time; nothing is interpolated.

2.3 Error measures

I use three measures. Throughout the paper, whiskered quantities are medians over random-initialisation seeds with whiskers spanning the minimum to the maximum seed.

1. **Relative L_2 error** (the primary measure): $\varepsilon_{\text{rel}} = \|y_\theta - y_{\text{ref}}\|_2 / \|y_{\text{ref}}\|_2$, with the norms taken over all evaluation times (or held-out states) and both components together; y_θ denotes the network output and y_{ref} the reference. Explicitly, for a trajectory sampled at times $\{t_k\}$, $\varepsilon_{\text{rel}}^2 = \sum_k \|y_\theta(t_k) - y_{\text{ref}}(t_k)\|^2 / \sum_k \|y_{\text{ref}}(t_k)\|^2$. Read it as a fraction of the trajectory’s overall size: 10^{-3} means the prediction differs from the truth by about a tenth of a percent of that size, and a value near 1 means the prediction is as wrong as the trajectory is large.

2. **Scale-normalised pointwise error:** $|\hat{y}_i - y_i|/\max_t |y_{i,\text{ref}}|$ per component, well defined even where a component crosses zero; used for error maps and curves.
3. **Capped pointwise relative error:** the squared error normalised by the squared size of the true state and capped at one,

$$\rho = \min\left(1, \frac{\|y_{\text{ref}} - y_{\theta}\|^2}{\|y_{\text{ref}}\|^2}\right), \quad \|y_{\text{ref}}\| = \sqrt{y_{1,\text{ref}}^2 + y_{2,\text{ref}}^2}, \quad (2)$$

evaluated per point (a time on a trajectory, or a held-out state). A worked example fixes the reading: for truth $(2, 0)$ and prediction $(1.9, 0.1)$ the error vector is $(0.1, -0.1)$, so $\rho = (0.1^2 + 0.1^2)/(2^2 + 0^2) = 0.02/4 = 0.005$; the prediction is wrong by half a percent of the state’s squared size. “Capped” means the ratio is cut off at one: wherever the squared miss exceeds the squared size of the true state, the recorded value is exactly 1, read as one hundred percent wrong. The cap exists because the denominator can approach zero: near-zero true states would otherwise dominate every average, and a miss larger than the state itself carries no more information than “completely wrong”. The state-modulus denominator is also deliberate: a componentwise denominator is undefined whenever that component crosses zero (twice per period for each component, and exactly at the start, where $y_2 = 0$), whereas the state modulus vanishes only at the equilibrium. In practice the cap engages only on order-one failed fits and at evaluation states whose true image is at or near the origin. Two reporting rules follow. The *median* of ρ is robust and tracks fine structure; the *mean* of ρ (a mean squared error, since ρ is already a squared quantity) is dominated by capped points, each of which contributes $1/N$ to the mean over an N -point set, so the mean cannot resolve anything below that level and is quoted only together with the number of capped points. Because ρ is squared, $\sqrt{\rho}$ compares directly with relative L_2 values, and the two measures cross-check each other throughout (for example, the selected network of Section 4.7 has six-period $\rho = 2.0 \times 10^{-6}$, i.e. $\sqrt{\rho} \approx 1.4 \times 10^{-3}$, consistent with its independently computed chained relative L_2 of 2.0×10^{-3}).

3 The continuous-time formulation

3.1 Method

The source work considers equations $u_t + \mathcal{N}[u] = 0$ and defines the residual $r = u_t + \mathcal{N}[u]$ (written f in the source work; this paper keeps f for the right-hand side of an ordinary differential equation and writes the residual as r). It approximates u by a network and obtains the residual of the network by automatic differentiation, the exact, chain-rule differentiation of the network with respect to its own inputs. The loss is the equally weighted sum of two mean-squared terms: a data term over the N_u points where the solution is known (initial and boundary data), and a residual term over N_f sampled *collocation points*, collocation points being the classical term for locations at which an equation is enforced.

For an ordinary differential equation the transfer is direct, with three simplifications and one modification. There is no spatial coordinate: the network maps one input, time, to the two-component state $y_{\theta}(t)$. There is no boundary: the data term collapses to a single anchor, the requirement that the network at $t = 0$ equal the starting state. This is one anchored point in place of the 50–100 used by the source work’s demonstrations, which makes the anchor’s task correspondingly harder. No spatial derivatives exist either: the only differentiation left is the time derivative of the network, $\dot{y}_{\theta}(t)$, obtained by differentiating the network’s output with respect to its own input. Automatic differentiation therefore appears in two distinct roles that should not be conflated: derivatives of the loss with respect to the weights θ drive the optimiser, as in any network training, while derivatives of the network output with respect to its input

t build the residual itself. Both are exact chain-rule computations, with no finite differences anywhere. The residual and loss are the exact transcription of the source construction,

$$r(t) = \begin{pmatrix} \dot{y}_{1,\theta}(t) - y_{2,\theta}(t) \\ \dot{y}_{2,\theta}(t) - \mu(1 - y_{1,\theta}^2(t)) y_{2,\theta}(t) + y_{1,\theta}(t) \end{pmatrix}, \quad \mathcal{L} = \|y_\theta(0) - y^0\|^2 + \frac{1}{N} \sum_{i=1}^N \|r(t_i)\|^2, \quad (3)$$

with $y^0 = (2, 0)$ and $\{t_i\}$ the collocation times. The one deviation: time is mapped to $[-1, 1]$ inside the network, because hyperbolic-tangent layers condition badly on raw inputs of size 40. The formulation also inherits a structural limitation from the source work: the starting state enters through the loss and is baked into the trained weights, so each trained network serves exactly one initial condition.¹ The discrete-time formulation of Section 4 removes exactly this.

3.2 Collocation budget

Because the collocation budget is central to how the source work frames the limitations of this formulation, I fix it explicitly and generously. The density is 20 collocation points per unit time at every horizon, placed by one-dimensional Latin hypercube sampling (a stratified random design: one point per equal-width stratum, so the domain is covered without clustering). A longer horizon therefore receives proportionally more points and is never undersampled by construction: $N = 60, 140, 280, 540, 800$ collocation points at the five horizons of Table 1. For comparison, the source work’s continuous-time demonstrations use 20,000 collocation points (Schrödinger equation) and 10,000 (Burgers equation) over two-dimensional space–time domains. On this problem the entire budget costs fractions of a second to evaluate; every experiment below pays it in full. Whatever fails in Section 3.4 therefore does not fail for want of collocation points. Sections 3.7 and 3.8 then vary the density over a 64-fold range and the placement directly, and measure the effect of each.

3.3 Protocol

All continuous-time experiments share: a fully connected network of 4 hidden layers of 50 hyperbolic-tangent units, in double precision (until Section 3.6 varies the architecture). Written out, the network computes

$$h_1 = \tanh(W_1 t + b_1), \quad h_k = \tanh(W_k h_{k-1} + b_k) \quad (k = 2, \dots, 4), \quad y_\theta(t) = W_5 h_4 + b_5,$$

with the weight matrices and bias vectors W_k, b_k collectively denoted θ . The hyperbolic tangent is used because it is smooth everywhere, and smoothness matters here: the residual of Eq. (3) differentiates the network, so a kinked activation such as the rectifier would hand the residual a discontinuous derivative. Training horizons are $T \in \{3, 7, 14, 27, 40\}$, i.e. 0.45 to 6.0 periods of the limit cycle (dividing by the period 6.663287), bracketing one period from either side, with three random-initialisation seeds per horizon, so that a single initialisation cannot masquerade as the method’s verdict.² The optimiser is full-batch L-BFGS (a deterministic quasi-Newton method using the entire loss at every step, as in the source work) with strong Wolfe line search. Every iteration uses the full set of collocation points, so one iteration is one epoch. Training runs in restarts of 200 epochs, where a restart clears the optimiser’s stored curvature history and continues from the current weights; there are at most 6 restarts, so at most 1,200 epochs, and training stops early once two consecutive restarts improve the loss by less than 1%. This is a convergence criterion rather than a budget, and every plain-formulation failure below is a converged failure. Scoring is on the dense reference grid (spacing 0.01) with the measures of Section 2.3.

¹An expensive way to solve one initial-value problem, and an equally expensive way to solve each subsequent one.

²A seed fixes the random initial weights and nothing else; a run is otherwise deterministic, so every number in this paper reproduces bit for bit.

Table 1: Faithful reproduction of the continuous-time formulation, swept over the training horizon. N is the number of collocation points (density fixed at 20 per unit time). Median, best and worst are over three seeds; the loss is the converged training loss.

T	periods	N	rel. L_2 (median)	best / worst seed	final loss
3	0.45	60	3.8×10^{-5}	$2.3 \times 10^{-5} / 4.6 \times 10^{-5}$	$1.4\text{--}2.0 \times 10^{-7}$
7	1.05	140	6.6×10^{-4}	$4.8 \times 10^{-4} / 3.9 \times 10^{-3}$	$2.3 \times 10^{-6}\text{--}2.8 \times 10^{-5}$
14	2.10	280	0.79	0.789 / 0.805	$\approx 6 \times 10^{-2}$
27	4.05	540	0.90	0.903 / 0.905	$\approx 3 \times 10^{-2}$
40	6.00	800	0.94	0.936 / 2.09	$2.1\text{--}4.8 \times 10^{-2}$

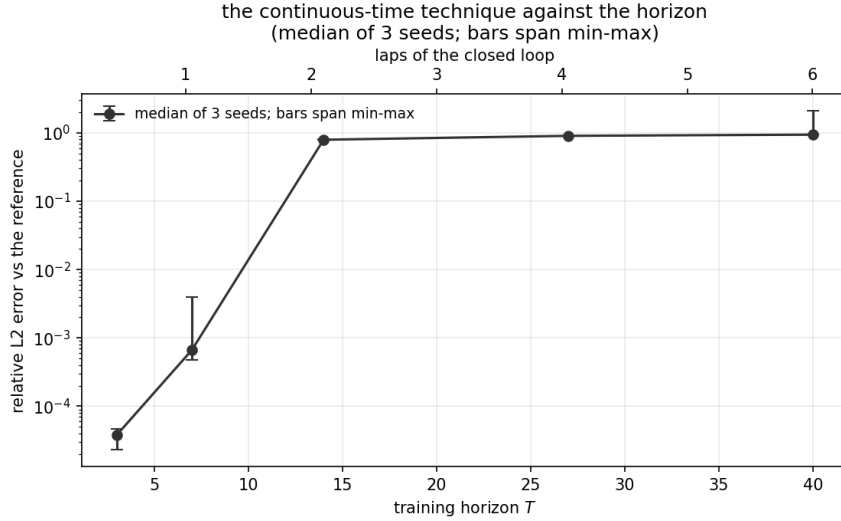


Figure 2: Relative L_2 error against training horizon for the faithful reproduction; median of three seeds with min-max whiskers. The formulation is excellent below one period of the limit cycle and its error is of order one beyond it.

3.4 Results: faithful reproduction

Table 1 and Figure 2 give the central result of the reproduction. Below one period the formulation is excellent: at $T = 3$ the median relative L_2 error is 3.8×10^{-5} , at or better than the accuracy the source work reports for its own continuous-time demonstrations (1.97×10^{-3} for the Schrödinger equation, 6.7×10^{-4} for the Burgers equation). This is unsurprising for a two-state oscillator against partial differential equations, but it confirms that the machinery is correctly built. Beyond one period the error is of order one at every horizon and every seed, with the seeds agreeing to two decimal places: the failure is systematic, not initialisation dependent.

The mechanism is visible in Figures 3 and 6. The origin satisfies Eq. (1) exactly, so a network that parks there pays no residual at all. The true dynamics repel from the origin, but a pointwise residual contains no stability information, so the loss cannot distinguish the attracting truth from the repelling impostor. The residual cost of escaping the true trajectory is confined to a short window of fixed size, and the mean over collocation points divides that fixed cost by the horizon, so the impostor becomes cheaper as T grows. At $T = 3$ the escape would corrupt most of the domain and the truth wins; at $T = 14$ it costs one period out of two and every seed falls. One $T = 40$ seed found a different escape, the 2.09 outlier in Table 1.³

Two findings follow. First, the failure is silent: the final losses converge to values of order 10^{-2} (Table 1) while the error is of order one, so a small, converged training loss is not evidence of

³Instead of settling at the origin it drifted outward to $y_1 \approx 4.7$, well outside the loop, and settled on a different wrong curve: the same mechanism with a different destination.

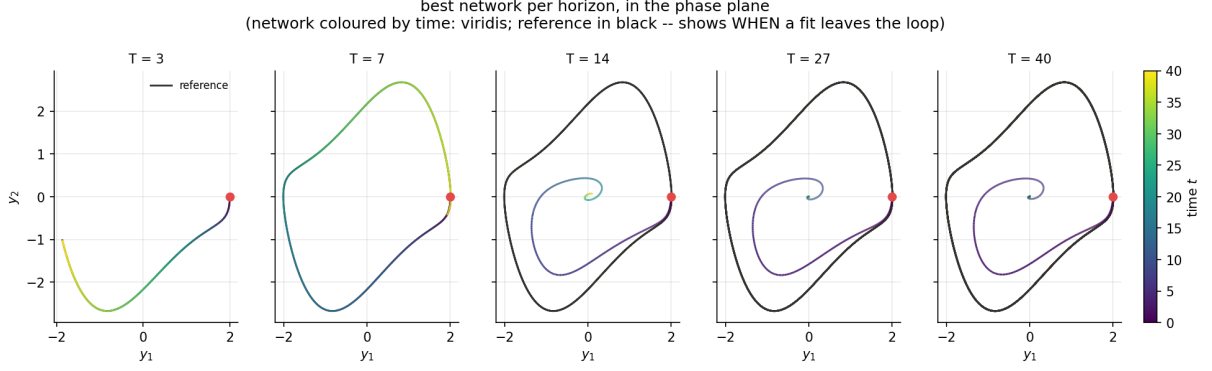


Figure 3: The failed fits in the phase plane (best seed per horizon). Past one period the fit abandons the closed loop and spirals into the origin, the repelling equilibrium, against the direction of the true flow.

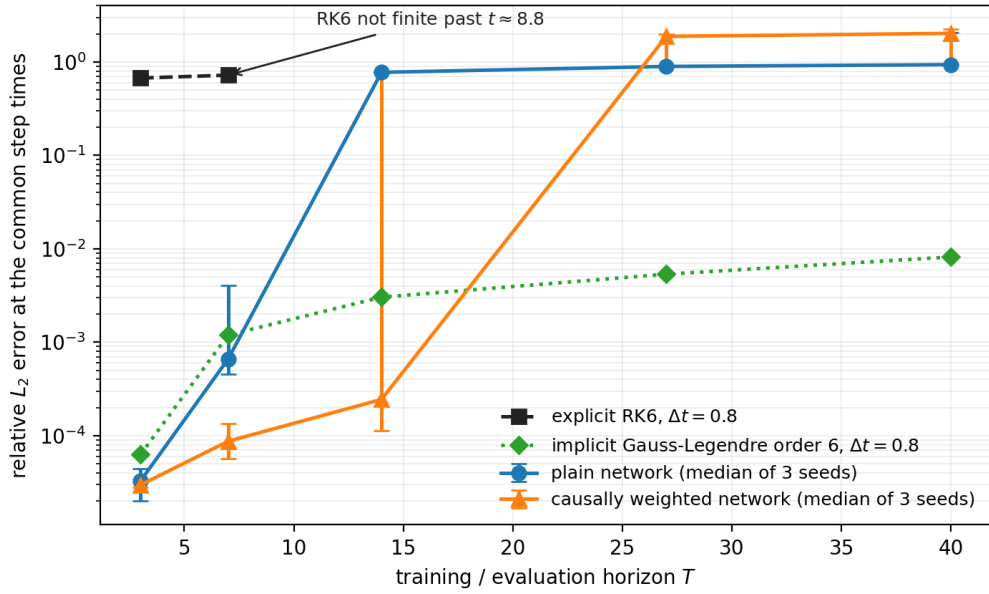


Figure 4: A classical anchor for the comparison: the two order-6 methods at the discrete-time study's step $\Delta t = 0.8$ from the same start, against the plain and causally weighted networks. The explicit method is the reference integrator's own seven-stage tableau; the implicit one is Gauss–Legendre with $q = 3$ (also order 6), solved by root-finding. All four are scored identically: relative L_2 against the reference over the common step times $\{0, 0.8, 1.6, \dots\} \leq T$ (network medians over three seeds with min–max whiskers; the classical runs are deterministic). The explicit method at this step is order-one wrong from the first horizon and its state is not finite past $t = 8.8$, so it has no value at $T \geq 14$; the implicit method is stable at every horizon.

Table 2: The comparison of Figure 4 as numbers: relative L_2 error at the common step times $\{0, 0.8, \dots\} \leq T$. The two classical columns are the explicit and implicit members of the same order-6 family at the same step; both runs are deterministic. Network entries are medians over three seeds. The step times differ from the dense scoring grid of Table 1, which is why the network values differ slightly from that table.

T	explicit RK6	implicit GL, order 6	plain network	causally weighted network
3	0.67	6.3×10^{-5}	3.3×10^{-5}	3.0×10^{-5}
7	0.72	1.2×10^{-3}	6.6×10^{-4}	8.7×10^{-5}
14	not finite	3.0×10^{-3}	0.78	2.5×10^{-4}
27	not finite	5.4×10^{-3}	0.90	1.88
40	not finite	8.2×10^{-3}	0.94	2.03

the two order-6 schemes at $\Delta t = 0.8$ in the phase plane (explicit steps coloured by time; implicit GL3 as diamonds; reference in black)

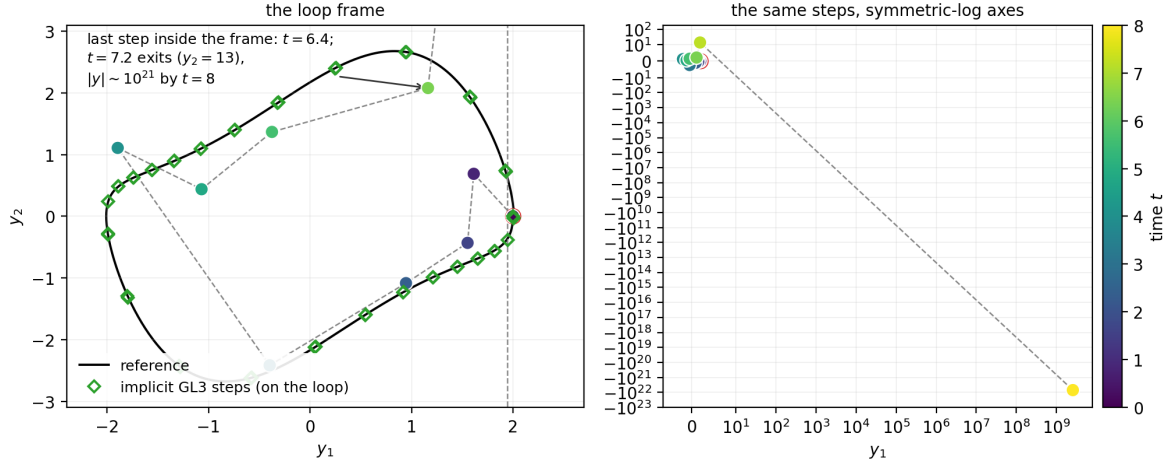


Figure 5: The classical runs of Figure 4 in the phase plane, drawn as in Figure 3: reference in black, the explicit RK6 steps coloured by time, the implicit Gauss–Legendre steps as open diamonds, the start marked in red. Left: the implicit steps sit on the loop all the way around, while the explicit steps sit off it from the outset (the first explicit step already reverses the sign of the velocity, $y_2 = +0.69$ where the reference has -0.68) and leave the frame after $t = 6.4$. Right: the explicit steps on symmetric-log axes, which is the only way to show the final excursion to $(2.6 \times 10^9, -7.1 \times 10^{21})$ at $t = 8$ on one pair of axes. Where the networks of Figure 3 leave the loop inward, toward the equilibrium, the explicit method at this step leaves it outward and does not return; the implicit method at the same step does not leave it at all.

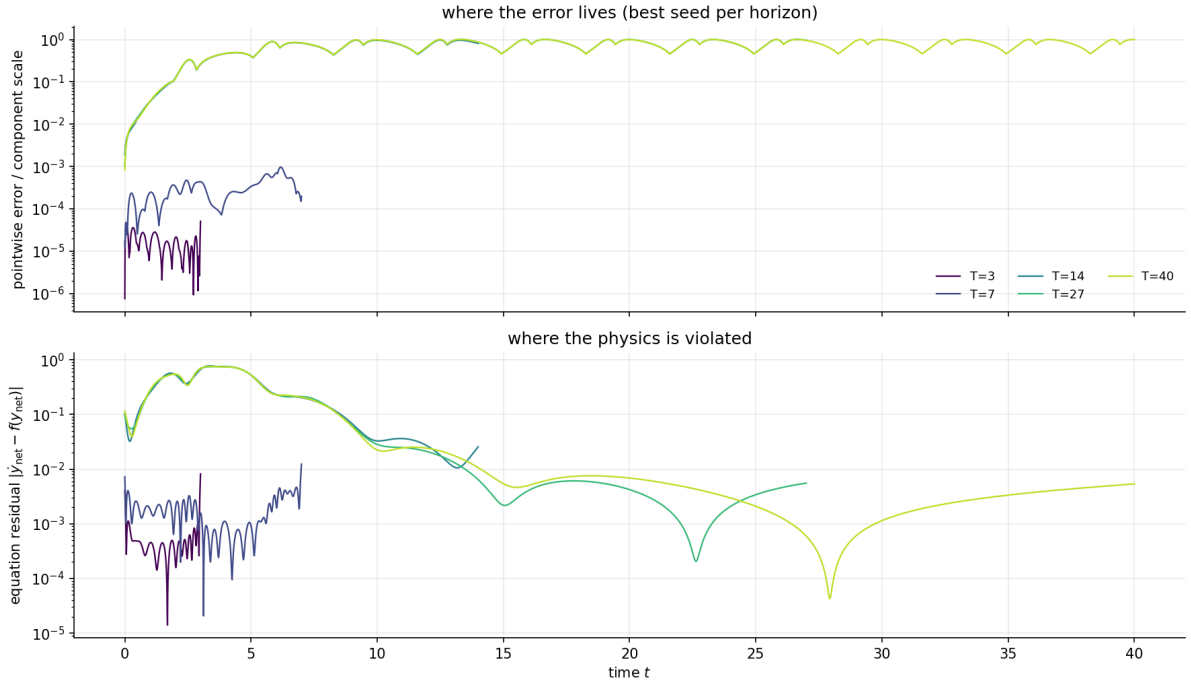


Figure 6: The failure mechanism. Top: where the error lives in time (logarithmic scale); at the horizons past one period the error leaves the anchor level within the first half period, between $t \approx 1.5$ and 3. Bottom: the equation residual along the network’s own trajectory. Past one period the residual is paid only in a short escape window and is negligible ever after: the physics is satisfied, on the wrong solution.

a correct solution. Second, the failure runs against the source work’s stated expectation that, for a well-posed problem with a unique solution, good accuracy follows from a sufficiently expressive network and sufficiently many collocation points. The problem here is well-posed with a unique solution, the identical network fits $T = 7$, and the collocation budget is trivially affordable and fully paid (Section 3.2), yet the method returns the wrong solution with a converged loss. What the loss lacks is anchoring: information about which solution is wanted, spread across the domain instead of concentrated at a single point. Collocation coverage is not the missing ingredient.

A classical integration at a comparable budget gives the comparison an anchor (Figure 4 and Table 2; the causally weighted variant shown there is introduced in Section 3.5, and its values are included so the comparison sits in one place). Two classical runs are shown, the explicit and implicit members of the same order-6 family at the same step $\Delta t = 0.8$, the step the discrete-time study of Section 4 uses: the reference integrator’s own seven-stage explicit tableau, and Gauss–Legendre with $q = 3$ (also order 6), solved by root-finding. Both are chained from the same start and scored by the same relative L_2 at the common step times. The explicit method touches f at seven points per step (21 evaluations by $T = 3$, 350 if run to $T = 40$), against the networks’ 20 collocation points per unit time re-evaluated at every epoch (up to 1,200 epochs for the plain variant, up to 61,200 including the Adam phase of Section 3.5); the implicit method adds a root-finding solve per step. The findings separate three cases. The explicit method at this step is unusable: a step of 0.8 is far outside its stable range on this problem, its state grows to $\sim 10^{21}$ by $t = 8.0$ and is not finite past $t = 8.8$, and where the networks work they are about four orders of magnitude below it (3.3×10^{-5} against 0.67 at $T = 3$). The implicit method of the same order at the same step is stable at every horizon, from 6.3×10^{-5} at $T = 3$ to 8.2×10^{-3} at $T = 40$; beyond two periods, where every network fails, it is the only method in the table that still works. And within the causally weighted variant’s working range it is the network that is more accurate for its single trained start: 3.0×10^{-5} against 6.3×10^{-5} at $T = 3$, 8.7×10^{-5} against 1.2×10^{-3} at $T = 7$, and 2.5×10^{-4} against 3.0×10^{-3} at $T = 14$. Figure 5 shows both runs in the phase plane, in the manner of Figure 3: the implicit steps sit on the loop throughout, while the explicit steps sit off it from the outset and leave the frame outward, where the networks’ failures collapse inward. The comparison therefore separates two different questions: at a fine step (the reference’s $h = 10^{-3}$) classical integration is better than every network in this paper by orders of magnitude, but at the coarse step $\Delta t = 0.8$ only implicit structure takes the step accurately, solved classically as here or learned as in Section 4.

3.5 Causal weighting and its limits

3.5.1 Method

The diagnosis above is that the loss fits all times simultaneously: late times settle onto a spurious solution before any information from the single anchor at $t = 0$ can reach them, because nothing in Eq. (3) encodes that an initial-value problem is determined forward in time. Causal weighting [4] encodes exactly that and nothing else. The residual at collocation time t_i is weighted by

$$w_i = \exp\left(-\varepsilon \int_0^{t_i} \|r(s)\|^2 ds\right), \quad (4)$$

with the integral evaluated as a left Riemann sum over the sorted collocation times $t_{(1)} < t_{(2)} < \dots$: writing $\Delta t_{(j)} = t_{(j)} - t_{(j-1)}$,

$$\int_0^{t_i} \|r(s)\|^2 ds \approx \sum_{j: t_{(j)} < t_i} \|r(t_{(j)})\|^2 \Delta t_{(j)}.$$

The time-weighting of the sum is deliberate: because ε multiplies a time-integral rather than a bare sum over points, its meaning is independent of collocation density, and doubling the

number of points does not double the exponent. Two different quantities scale differently here, and it is worth separating them: the normalisation keeps the meaning of ε independent of how many points share the domain, but the value of the integral itself still grows with the length of trajectory being fitted, so conditions on the weights become harder to meet as the horizon grows. This matters for the budget-extension test below and is taken up again in the Discussion. The weights are recomputed from the current residuals at every optimiser iteration but treated as fixed constants by the gradient (detached from the differentiation), so the optimiser never gains by manipulating its own weighting. A collocation point counts only once the trajectory *before* it already satisfies the equation: a front of solved trajectory propagates outward from the anchor, and the far domain, where the weights are still near zero, cannot be captured by a spurious solution in the meantime. On convergence all residuals vanish, every weight rises to 1, and the objective becomes exactly the plain loss, so the weighting removes itself over the course of a successful training.

The weighting is incompatible with L-BFGS, which requires an objective that does not change between iterations (L-BFGS builds its curvature estimate by comparing gradients across its history, and a loss whose weights move under it invalidates that comparison). I measured two hybrid schedules, and both fail:

- freezing the weights per L-BFGS restart lets the front advance only at the six refreezes; it stalled 42% of the way across $T = 14$ (relative error 2.15);
- splitting the same 1200 epochs into 40 short restarts leaves each restart too short to move the front; it oscillated around 30% (relative error 1.58).

The front must move continuously; per-iteration weight updates are part of the method. The algorithm actually used therefore follows the causal-weighting authors:

1. an Adam phase, Adam being a first-order gradient method with per-parameter step sizes (learning rate 10^{-3}); each step uses the full batch, so steps and epochs coincide, and the weights are recomputed from detached residuals at every epoch;
2. ε raised through the ladder $\{0.01, 0.1, 1, 10, 100\}$ (small ε tolerates upstream error; large ε demands it be gone), advancing one rung whenever every weight exceeds 0.99, all within a limit of 60,000 Adam epochs; and
3. the same L-BFGS optimisation of Section 3.3 on the plain loss, once every weight is near 1.

I write *the weight condition* for the requirement that every weight exceed 0.99: it is the schedule’s advance condition and, at the final ε rung, the working sign that the front has crossed the whole domain. Causal weighting thus acts as a warm start that delivers L-BFGS into the right basin, and both variants of every comparison end by optimising the same objective with the same optimiser. Two quantities are logged throughout as telemetry: the minimum weight, and the fraction of collocation points with weight above one half, written ϕ below.

3.5.2 Results

Table 3 and Figure 7 summarise the comparison. At $T = 14$ (two periods, where the plain formulation fails at 0.79) causal weighting reaches 2.1×10^{-4} , an improvement of three to four orders of magnitude; in the capped measure of Eq. (2) the contrast at $T = 14$ is about seven orders of magnitude (mean ρ over the trajectory: 6.2×10^{-1} for the plain fit against 4.0×10^{-8} for the causally weighted one, computed from the saved run outputs). The cost of crossing the domain grows faster than linearly: roughly 3.5k Adam epochs at half a period, 10–30k at one period, 60k at two, and beyond the limit at four and six periods, where runs stopped mid-domain score *worse* than the plain variant because the far part of the domain, its weights still near zero,

Table 3: Plain versus causally weighted training at equal collocation density and architecture; medians over three seeds. The two variants differ in budget as well as weighting: the plain variant trains for at most 1,200 full-batch L-BFGS epochs (a convergence criterion that always fired), while the causally weighted variant adds up to 60,000 full-batch Adam epochs before an identical final L-BFGS phase.

T	periods	plain	causal	front progress (Adam epochs)
3	0.45	3.8×10^{-5}	4.8×10^{-5}	weight condition met, 3–5.5k
7	1.05	6.6×10^{-4}	7.6×10^{-5}	weight condition met, 8.5–28.5k
14	2.10	0.79	2.1×10^{-4}	epoch limit reached; two seeds near 2×10^{-4} , third at 0.71
27	4.05	0.90	1.90	epoch limit reached mid-domain, all seeds (0.88/1.90/1.99)
40	6.00	0.94	2.04	epoch limit reached mid-domain, all seeds (0.95/2.04/2.27)

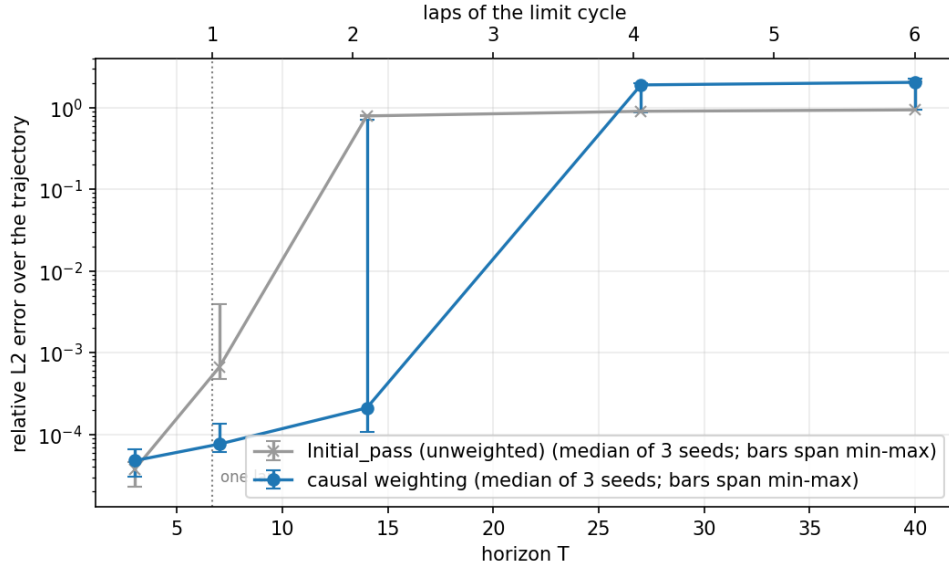


Figure 7: Same metric, reference and collocation density on both sides; the causally weighted runs additionally use the Adam phase described in the text. Median of three seeds with min-max whiskers. Causal weighting moves the horizon at which the method fails from between one and two periods to between two and four.

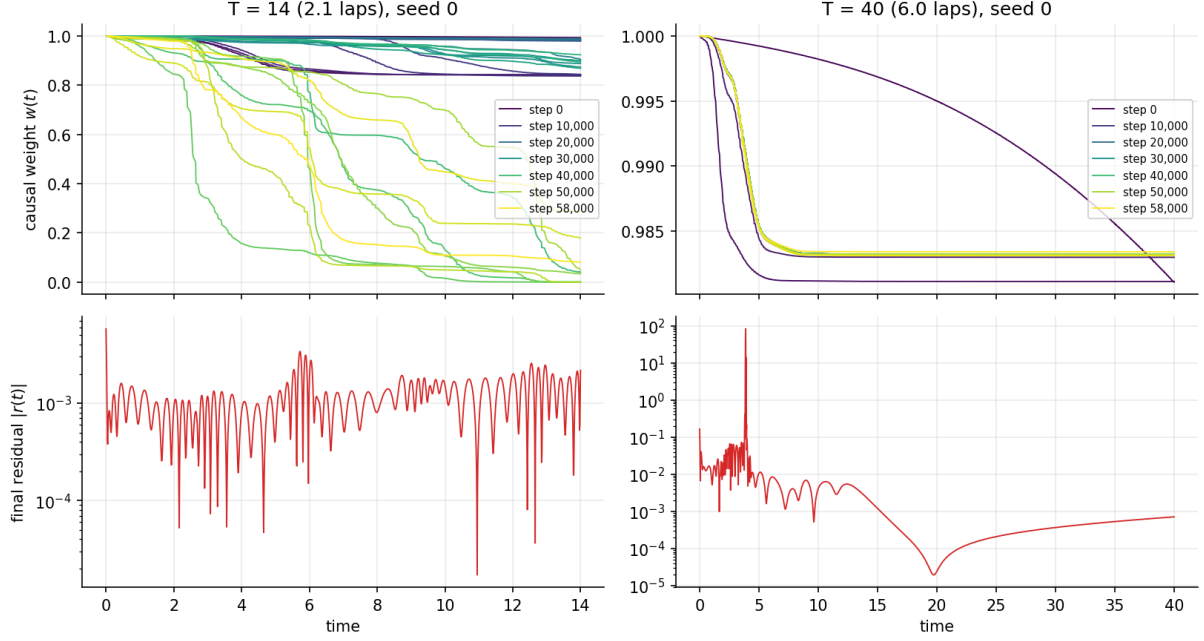


Figure 8: The mechanism at work: snapshots of the weight profile $w(t)$ during training (colour: Adam epoch) at $T = 14$ and $T = 40$; below, the equation residual along the final fit. A front of large weights sweeps outward from the anchor as the trajectory behind it becomes consistent with the equation.

Table 4: The budget-extension test: the seven runs that reached the epoch limit, rerun at four times the budget. None of the seven completed the crossing.

run	60k epochs	240k epochs	telemetry reading
$T = 14$, seed 2	0.71	0.61	weight condition met at 92k; still wrong (jump between points)
$T = 27$, seed 0	0.88	0.83	front advancing, far slower than linearly in epochs
$T = 27$, seed 1	1.90	0.24	weights saturated at small $\varepsilon \sim 185$ k epochs; then ε advanced
$T = 27$, seed 2	1.99	1.99	no front formed; ϕ below 0.1 throughout
$T = 40$, seed 0	0.95	0.79	front advancing, far slower than linearly in epochs
$T = 40$, seed 1	2.04	2.08	weights saturated at small ε for the whole run
$T = 40$, seed 2	2.27	2.29	no front formed

is unconstrained. The essential difference from Section 3.4 is that these failures are visible in the training telemetry: the minimum weight and ϕ show the front still mid-domain. The plain failures, by contrast, reported themselves as converged. One of the runs that ended at the epoch limit also produced a second kind of silent failure, after the collapse onto the equilibrium of Section 3.4: a fit with training loss 9.7×10^{-6} and error 0.71, an out-of-phase near-solution joined to the truth by a jump inside an unsampled interval between collocation points, invisible both to the loss and to weights computed on the same grid. Sections 3.7 and 3.8 test directly whether more points, or better-placed points, change these outcomes.

3.5.3 The budget-extension test

Every causally weighted run at $T \geq 14$ ended at the 60,000-epoch limit with its front mid-domain. So does the long-horizon failure come down to the epoch budget? The test is direct: rerun exactly those seven runs at 240,000 epochs, four times the limit, with nothing else changed.

Table 4 answers the question: the original limit was genuinely too small for some runs (one needed 92k epochs to meet the weight condition), but four times the budget completed the crossing in none of the seven. The telemetry (Figure 9) separates three outcomes, and the phase portraits (Figure 10) show what each looks like as a trajectory. In the first, the front advances

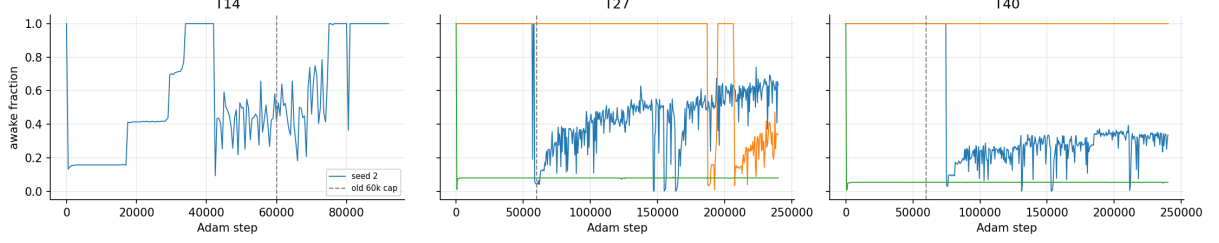


Figure 9: The discriminating instrument: ϕ , the fraction of collocation points with weight above one half, over the extended runs, with the original 60k-epoch limit marked. The three outcomes described in the text are directly visible: a front advancing too slowly to finish; weights saturated at small ε (note the $T = 27$ transition near 185k epochs, where ε finally advances and the error begins to fall); and no front forming at all, ϕ flat near zero.

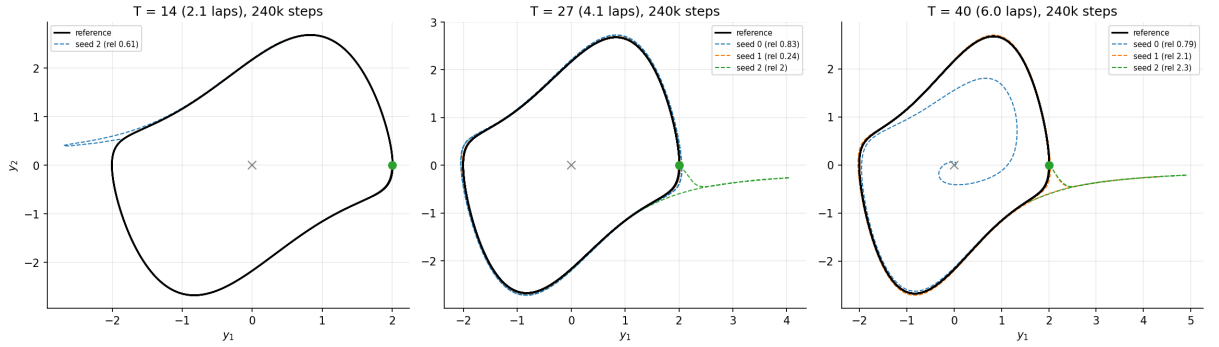


Figure 10: Every extended-budget run in the phase plane. The runs whose weights saturated early ride the reference loop almost perfectly and are wrong *in time*: they circulate out of phase, so the trajectory error is of order one while the orbit looks flawless. The runs where no front formed satisfy the anchor and then leave the loop. The one run that met the weight condition traces the loop with a single excursion, the jump between collocation points described in the text.

Table 5: Success by starting state: causally weighted variant at $T = 14$ (two periods), counting a run a success at relative $L_2 < 10^{-2}$, out of the 25 architectures per start. $\|y^0\|$ is the start’s distance from the equilibrium at the origin; $(2, 0)$ lies on the limit cycle.

start (y_1, y_2)	$\ y^0\ $	successes / 25	best run
$(2.00, 0.00)$	2.00	9	6.7×10^{-5}
$(1.85, 2.90)$	3.45	6	1.1×10^{-3}
$(-2.32, -2.79)$	3.63	3	1.5×10^{-3}
$(0.87, 1.76)$	1.96	1	3.5×10^{-4}
$(-0.03, -1.02)$	1.02	0	1.9×10^{-2}
$(-0.86, -0.24)$	0.89	0	0.91

but too slowly to finish within even the extended budget. In the second, the weight condition is never met at the first ε value, so no front ever forms. In the third, the weights saturate near 1 while ε is still small; the weighting then has little effect, training proceeds much as if unweighted, and the fit settles on a wrong solution. The one run of the third kind in which ε eventually advanced ($T = 27$, seed 1) improved to 0.24, still an order-one error, and the improvement coincided with the ε advance rather than with the added epochs. The binding constraints are structural: (i) the whole-history integral in Eq. (4) grows with the length of trajectory being fitted, so the weight condition becomes harder to meet as the horizon grows, and the schedule can leave a run with no front formed or with its weights saturated too early; and (ii) both the loss and the weights see only the collocation grid, so a jump inside an unsampled interval defeats even a run that met the weight condition. Neither constraint yields to epochs at fixed collocation density.

3.6 Network size and the starting state

The budget axis being closed, the next candidate resource is the network itself, and with it a variable every experiment so far had frozen: the starting state. I cross depth $\{2, 4, 6, 8, 10\}$ with width $\{16, 32, 64, 128, 256\}$ (25 architectures), with both variants (plain and causally weighted), two horizons ($T = 14$, where the plain variant first collapses; $T = 27$, the first horizon at which the causally weighted variant fails), and six starting states: $(2, 0)$ plus five Latin-hypercube-drawn states across the rectangle $y_1 \in [-2.5, 2.5]$, $y_2 \in [-3, 3]$ (the limit cycle with margin, and the same region the discrete-time formulation later trains over), one seed each: $25 \times 2 \times 2 \times 6 = 600$ runs. Each combination is trained once; the sweep trades seed replication for coverage of architectures and starts, so statements below are made at the level of 25-architecture groups or 150-run success rates, and any single entry carries the seed-to-seed variability measured at three seeds elsewhere in this paper.

No architecture repairs a failing horizon (Figure 11). In the plain variant at $T = 14$ every architecture fails, with per-architecture medians over the six starts of 0.82–0.99 and a best single run of 0.272 across all 150; at $T = 27$ the medians are 0.93–0.98. In the causally weighted variant at $T = 14$ there is no monotonic improvement with size (best entry: depth 2, width 64, median 0.151; most entries 0.4–1.2), and at $T = 27$ the outcome is flat: entry medians 0.86–1.22 and a best single run of 0.685. Nothing succeeds at four periods, at any size. Two readings hold across variants and horizons: added depth is actively harmful, and added width stops helping beyond 64–128 units.

So what does order the outcomes? The starting state (Table 5 and Figures 12 and 13). Across the whole rectangle, the causally weighted variant’s success rate at the 1% threshold is $19/150 = 12.7\%$ at two periods and $0/150 = 0\%$ at four; the plain variant’s is 0% at both, so at four periods none of the 300 runs of either variant succeeds. (At a looser threshold of relative error 0.1 the two-period count rises to 26 of 150, every one of them causally weighted; nothing else changes.) The two inner starts, closest to the equilibrium, never succeed; an inner start

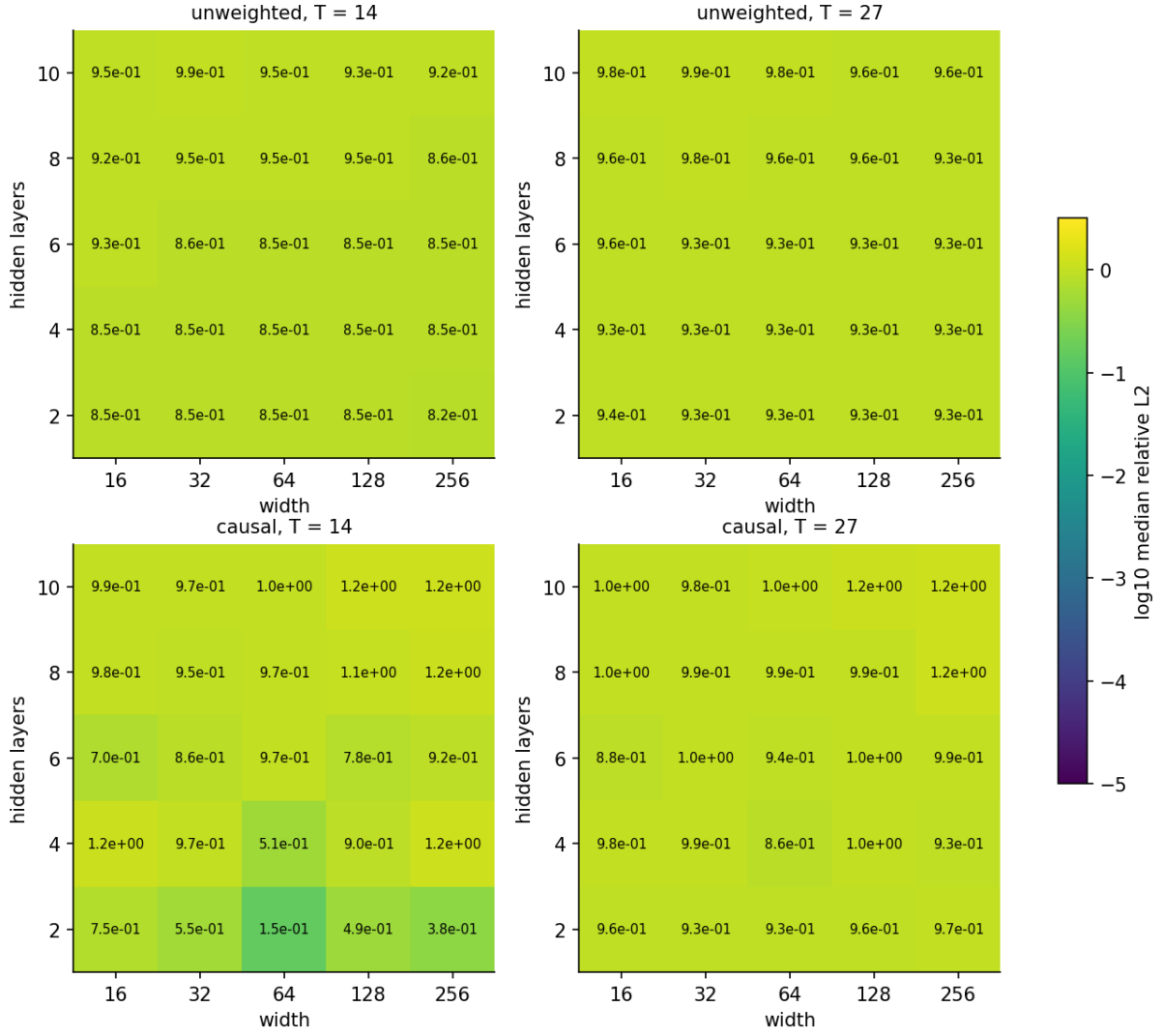


Figure 11: Median relative L_2 error by architecture (depth $\times$ width), one heatmap per variant and horizon, medians over the six starting states: no row or column repairs the failing horizons. Network size does not change where the method fails.

Causally weighted, $T = 14$ (two laps): the best of 25 architectures, per starting state

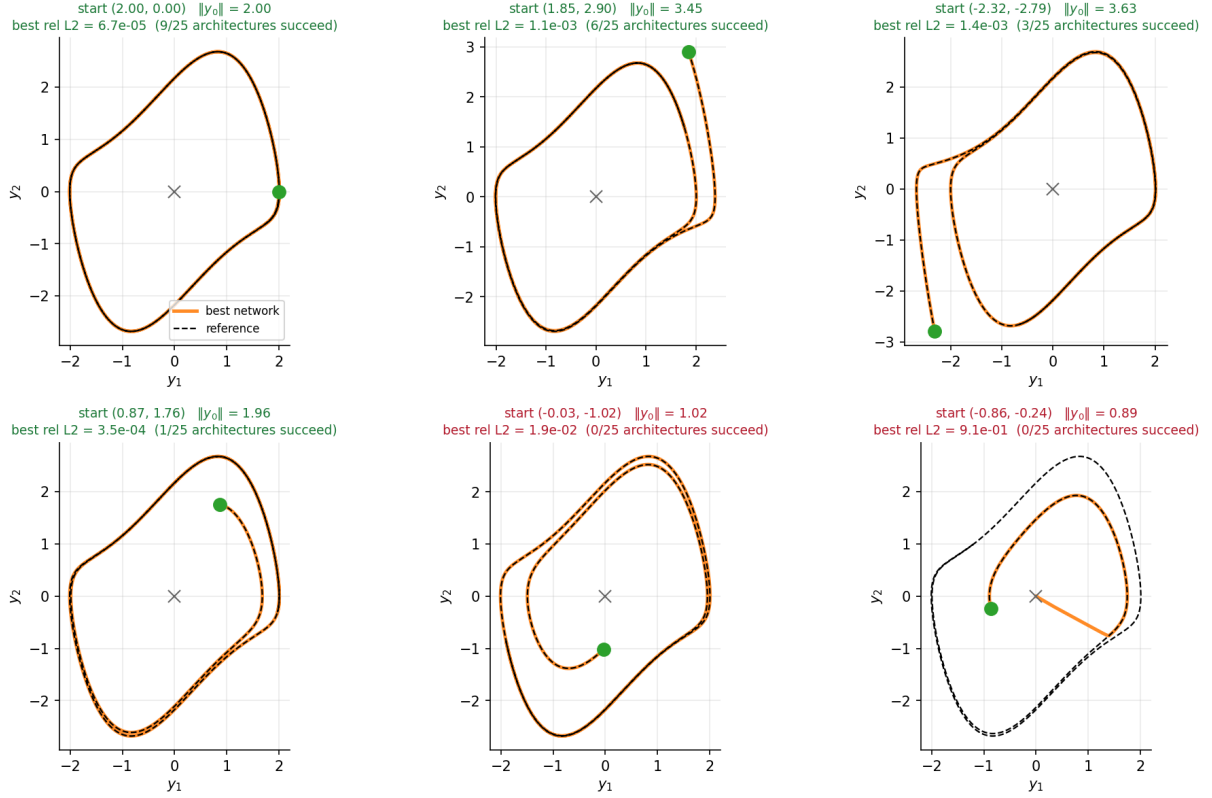


Figure 12: Best causally weighted $T = 14$ run (of 25 architectures) for each of the six starting states, in the phase plane: the outer starts track the loop; the two starts nearest the equilibrium are not solved by any architecture.

must first spiral outward past the zero-residual solution at the origin, which an outer start never approaches. The most successful start, $(2, 0)$, is the one lying exactly on the limit cycle; distance from the equilibrium and lying on the cycle are confounded in this set of six starts, so I report the counts without separating the two effects.

3.7 Collocation density

The remaining resource axes are the collocation points themselves: how many, and where. I cross densities $\{5, 80, 320\}$ points per unit time with horizons $T \in \{7, 14, 27, 40\}$, three seeds and both variants, 72 runs; the density-20 values are those of Sections 3.4 and 3.5. I placed two hypotheses on record before the sweep:

- H1: the plain variant is density-independent at the horizons where it has failed, because the mean in Eq. (3) spreads the escape window's fixed cost over however many points share the domain;
- H2: the causally weighted variant is density-sensitive, through the unsampled intervals between collocation points (Section 3.5).

H1 is confirmed at the horizons where the plain variant has failed (Table 6 and Figure 14): across a 64-fold density range the medians at $T = 14, 27, 40$ move by factors of 1.15, 1.01 and 1.003. Collocation cost was never the binding limit there, and this is now a direct measurement rather than an affordability argument. At $T = 7$, just past one period, the plain variant sits at the margin of its working range and the outcome is dominated by the seed rather than the

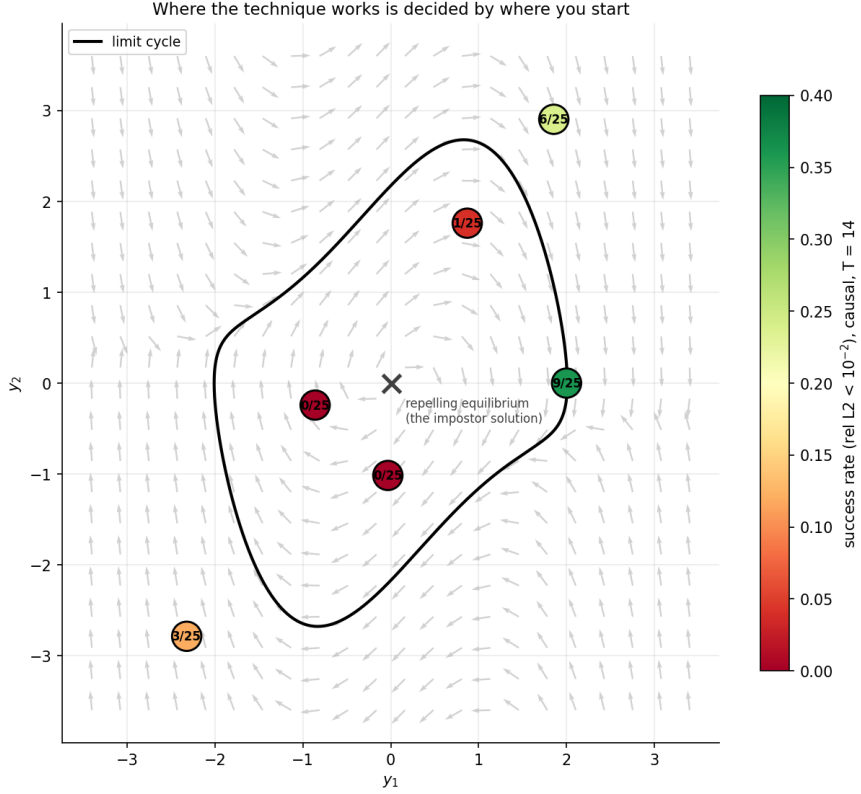


Figure 13: The six starting states drawn on the vector field, coloured by success count: the counts fall toward the equilibrium, and the start on the limit cycle is the most successful.

Table 6: Relative L_2 error against collocation density (points per unit time); medians over three seeds. “wc” marks runs in which the weight condition (every weight above 0.99) was met but the fit was wrong.

variant	T	$d = 5$	$d = 20$	$d = 80$	$d = 320$
plain	7	6.4×10^{-2}	6.6×10^{-4}	9.4×10^{-2}	2.1×10^{-3}
plain	14	0.913	0.793	0.790	0.792
plain	27	0.915	0.904	0.905	0.904
plain	40	0.943	0.942	0.942	0.941
causal	7	0.754 (wc 3/3)	7.6×10^{-5}	7.9×10^{-5}	9.7×10^{-5}
causal	14	0.974 (wc 2/3)	2.1×10^{-4} (worst 0.71)	1.8×10^{-4}	1.9×10^{-4}
causal	27	2.70	1.90	1.45	1.45
causal	40	2.89	2.04	2.05	2.05

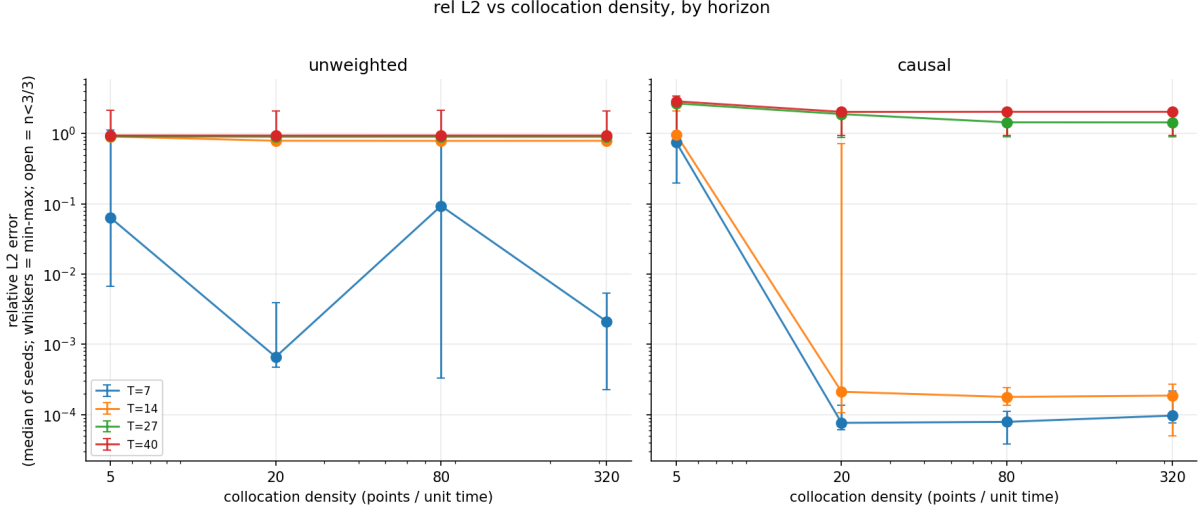


Figure 14: Relative L_2 error against collocation density (logarithmic axis), one line per horizon, plain (left) and causally weighted (right); medians with min-max whiskers. The plain lines are flat at the failed horizons; the causally weighted lines drop sharply between density 5 and 20 at the horizons the method can reach and stay flat beyond, and they stay high at the horizons it cannot.

density: the medians range from 6.6×10^{-4} to 9.4×10^{-2} with min-max spreads of up to three orders of magnitude, and the variation is not monotone in density. H2 refines into a threshold. Below roughly 20 points per unit time the weight condition stops being informative: at density 5 it is met within about 12k Adam epochs on wrong solutions (three of three seeds at $T = 7$), because with gaps this wide the weighting no longer orders the fit in time (Figure 15). At density 20 and above, the one-period error collapses to $\sim 10^{-4}$ and stays flat to 320, with a density-independent crossing cost, exactly as the ε normalisation of Eq. (4) intends. At two periods the picture across densities is: one of three seeds at density 20 finished at 0.71, while at 80 and 320 all seeds landed near 2×10^{-4} . The alternative placements of Section 3.8 also finish three of three at density 20, so the single failing run singles out neither the density nor the placement; I note only that every tested change removed it. At four and six periods density changes nothing: the causally weighted errors stay at 1.45–2.9 from 5 to 320 points per unit time.

3.8 Collocation placement

Two kinds of escape through unsampled intervals appeared above: the plain variant’s trajectories leave the reference level within the first half period (Figure 6, top), and one causally weighted run at $T = 14$ jumped inside an unsampled interval between collocation points. Both raise the question of whether *where* the points sit matters. I hold the density fixed at 20 and compare two alternative placements against the Latin-hypercube placement used everywhere else: *uniform* (deterministic bin midpoints) and *anchored* (Latin hypercube plus 40 extra points spaced geometrically inside $(10^{-3}, 1]$, i.e. successive spacings in fixed ratio, concentrating points just after the anchor), each crossed with both variants, $T \in \{14, 27\}$ and three seeds: 24 runs.

The plain variant is placement-independent at both horizons (medians 0.786–0.806 at $T = 14$, 0.901–0.910 at $T = 27$, indistinguishable from the Latin-hypercube placement). In the causally weighted variant at $T = 14$, uniform and anchored placements both finish three of three seeds at $\leq 3 \times 10^{-4}$ where the Latin-hypercube placement finished two of three; a single differing run out of three cannot separate a placement effect from seed-to-seed variability, and I report the counts as they stand. At $T = 27$ anchoring does not change the four-period outcome: best anchored seed 0.900 (others 0.900 and 1.96); uniform 0.901, 2.85, 1.99 (Figure 16). Concentrating points in the interval the escapes were seen to use does not prevent the failure, which suggests the

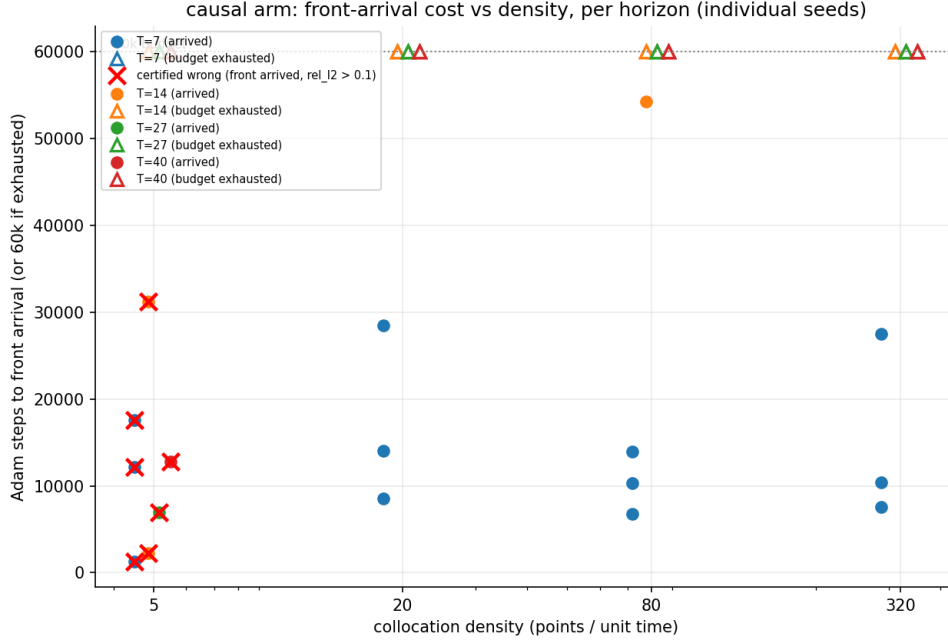


Figure 15: The Adam epochs needed to meet the weight condition, against density, with the wrong-solution cases marked: at density 5 the condition is met cheaply on wrong solutions; at density 20 and above the cost of meeting it is density-independent.

escape is a symptom of a front that never crossed rather than the cause of the failure. Together with Section 3.7 this measures the collocation axis: neither how many points nor where they sit changes the long-horizon outcome.

3.9 What the continuous-time route establishes

The continuous-time formulation works, per starting state, up to a horizon that no resource moves. It reproduces the source work’s accuracy below one period; causal weighting, the strongest loss-level modification in the literature, moves the failure from one period out to two and makes it visible in the training telemetry; and beyond that, optimisation budget ($4\times$, none of seven), network size (600 runs, depth harmful), collocation density (64-fold range) and collocation placement all leave the outcome unchanged. The one variable that orders outcomes is the starting state: success at two periods occurs in 19 of 150 causally weighted runs at the 1% threshold and depends on where the start sits, and at four periods none of the 300 runs of either variant succeeds. Taken together, these findings motivate imposing the physics structurally instead, which is the discrete-time formulation of the next section.

4 The discrete-time formulation

4.1 Implicit Runge–Kutta background

Integrating $\dot{y} = f(y)$ exactly across one step gives

$$y(t + \Delta t) = y(t) + \int_t^{t+\Delta t} f(y(s)) ds, \quad (5)$$

so a one-step numerical method is, at heart, a quadrature rule (a recipe for approximating an integral from a few point evaluations) applied to Eq. (5). The simplest choice, approximating the integrand by its value at the left endpoint, is Euler’s method, $y^{n+1} = y^n + \Delta t f(y^n)$. Better quadrature needs the integrand at interior times of the step; but the integrand is $f(y(s))$, and

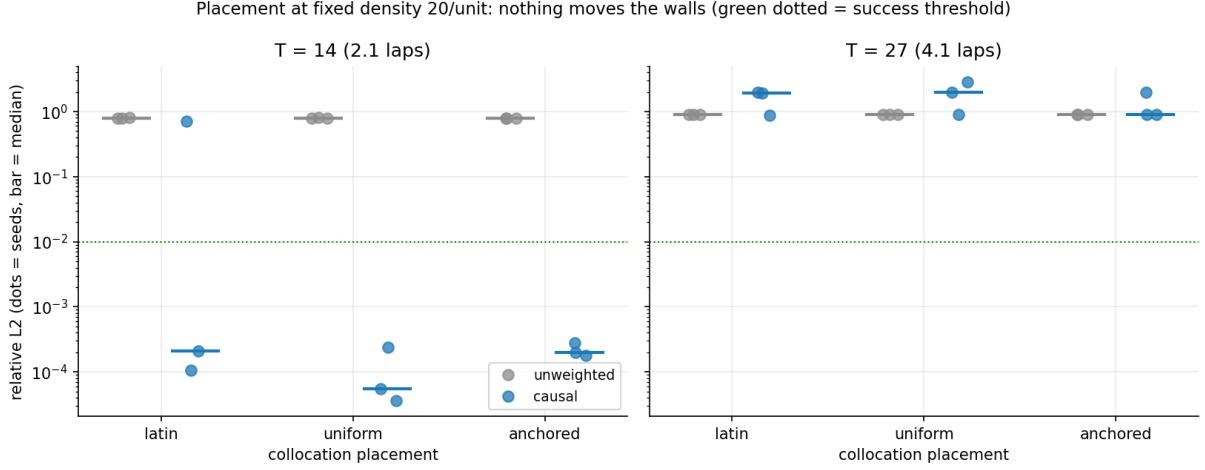


Figure 16: The placement comparison at both horizons and variants: three placements, indistinguishable outcomes.

$y(s)$ is exactly what is not yet known there. The Runge–Kutta family resolves this circularity by introducing q *stage states*, internal estimates of the solution at chosen fractions $c_1, \dots, c_q$ of the step, defined self-consistently through each other:

$$y^{n+c_j} = y^n + \Delta t \sum_{k=1}^q a_{jk} f(y^{n+c_k}), \quad y^{n+1} = y^n + \Delta t \sum_{j=1}^q b_j f(y^{n+c_j}), \quad (6)$$

where y^{n+c_j} denotes the stage state at time $t_n + c_j \Delta t$, and the nodes c , matrix $A = (a_{jk})$ and weights b form the Butcher tableau, which specifies the method completely. If A is strictly lower triangular, each stage depends only on earlier ones and the step is computed left to right: an explicit method (the sixth-order reference integrator of Section 2 is one). When A is full, every stage depends on every other and the first relation of Eq. (6) is a genuine coupled system of $2q$ nonlinear equations (two state components times q stages): the method is implicit, normally solved by a root-finder at every step.

Why pay the implicit price? Two properties, and both matter here.

- **Order.** A method has order p if the error of one step shrinks like Δt^{p+1} , so that the accumulated error over a fixed interval shrinks like Δt^p ; halving the step of an order-6 method cuts the error 64-fold. Gauss–Legendre tableaus place the nodes c_j at the roots of the degree- q Legendre polynomial mapped to $[0, 1]$, the same points that make Gaussian quadrature exact for polynomials of degree $2q - 1$, and attain order $2q$, the maximum any q -stage method can achieve: each added stage buys two orders.
- **A-stability.** Stability is probed on the linear test equation $\dot{y} = \lambda y$ with $\text{Re } \lambda < 0$, whose true solution decays; a method is A-stable if its numerical solution also decays at every step size. Explicit methods never are (too large a step and the numerical solution explodes although the true one decays), while Gauss–Legendre methods are A-stable at every order [6]. A-stability is, however, a *linear* property: on a nonlinear system the implicit stage equations must also have a findable solution, and that solvability, not stability, is what breaks first as the step grows (measured directly in Section 4.8).

Gauss–Legendre methods are also collocation methods in the classical sense: there is a single degree- q polynomial through y^n that satisfies the differential equation exactly at the q node times, and the stages are its values there, so the stage states approximate the solution at the interior times $c_j \Delta t$ to high order. They are not abstract intermediates (Figure 17).

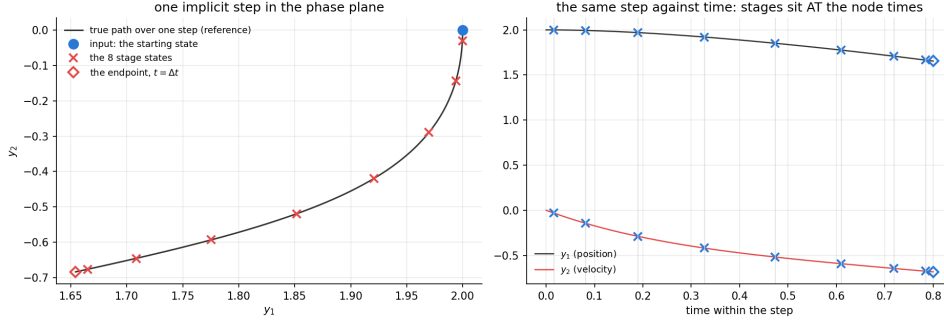


Figure 17: Anatomy of one implicit step from $(2, 0)$ at $q = 8$: the true path over $[0, 0.8]$, the eight Gauss stage states (exact solver), and the endpoint. The stages lie on the true mini-trajectory, crowding toward the ends of the step where the Legendre roots sit. Given the input state, the network must place all nine points.

4.2 The loss

The source work’s discrete-time construction replaces the root-finding solve of Eq. (6) by a network that proposes all $q + 1$ unknowns at once; each defining relation is rearranged to isolate the known state,

$$\underbrace{y^{n+c_j} - \Delta t \sum_k a_{jk} f(y^{n+c_k}) = y^n}_{\text{reconstruction } j} \quad \text{and} \quad \underbrace{y^{n+1} - \Delta t \sum_j b_j f(y^{n+c_j}) = y^n}_{\text{reconstruction } q+1}, \quad (7)$$

so that if the network’s outputs really are the stage states of the scheme, all $q + 1$ reconstructions land exactly on y^n . The loss I use is the mean of the squared reconstruction mismatches over outputs, components and batch. (In the source work the loss is a sum rather than a mean and carries an additional boundary term; for an ordinary differential equation there is no boundary, and at fixed batch size the mean is a rescaled sum. Both differences are noted for exactness.) Nothing else is supervised: there is no time derivative of the network, no collocation grid over time, and no trajectory data. The tableau is fixed and known; only the stage values are learned. The anchoring that the continuous-time formulation concentrates in a single point is here built into every term of the loss: every output is tied to a known state through the algebra of the step.

4.3 The state as the input: a reusable one-step map

In the source work the network input is a spatial coordinate and the known y^n is a measured snapshot of a field; an ordinary differential equation has no spatial coordinate, so the input becomes *the starting state itself*: the network maps (y_1, y_2) to the q stage states plus the endpoint, $2(q + 1)$ numbers. Trained over a region of starting states, one network becomes a reusable map for the fixed step: feed in any state and receive the state $\Delta t = 0.8$ later; feed the endpoint back in and a whole trajectory follows, without retraining. The step size 0.8 is taken from the source work’s own demonstration, so the comparison inherits its setting; no other value is tested here, a limitation Section 6 takes up. This construction differs from the sequential mode described in the source work, which retrains at every step. The generalisation itself follows Fernández Corral et al. [5], who apply it to conservative and periodically forced systems; the dissipative limit-cycle instantiation, the exact-scheme error attribution, the seed and selection analysis, and the head-to-head comparison with the continuous-time formulation below are, to my knowledge, new.

Table 7: Endpoint relative L_2 error on the 441 held-out states versus the number of stages q (median over three seeds), against the exactly solved scheme on the same states and metric.

q (order $2q$)	trained network (median)	exact scheme, same grid
2 (4)	3.94×10^{-2}	4.20×10^{-2}
4 (8)	4.57×10^{-3}	2.20×10^{-4}
8 (16)	3.61×10^{-3}	2.38×10^{-8}
16 (32)	3.63×10^{-3}	8.9×10^{-14}
32 (64)	3.63×10^{-3}	1.4×10^{-13}

4.4 Protocol and guardrails

I train on 2000 starting states per run, Latin-hypercube sampled over the rectangle $y_1 \in [-2.5, 2.5]$, $y_2 \in [-3, 3]$, the limit cycle with margin. The rectangle is a sampling region only; nothing constrains a trajectory to remain inside it, unlike a boundary condition in a partial differential equation. (This is the discrete-time analogue of the collocation budget of Section 3.2: the equations are enforced at 2000 points of the *state space*, rather than on a grid in time; Section 4.9 varies this count and finds the error flat from about 500 points.) The network body is identical to the continuous-time experiments (4 hidden layers of 50 hyperbolic-tangent units, double precision) with a linear output layer of $2(q + 1)$ units; only the output layer grows with q . At $q = 8$ this means 18 outputs and 8,718 trainable parameters, and the interior node times of the step are $c_j \Delta t = \{0.016, 0.081, 0.190, 0.327, 0.473, 0.610, 0.719, 0.784\}$, the Gauss–Legendre nodes of Section 4 scaled by $\Delta t = 0.8$. Each input component is scaled by its half-range (y_1 by 2.5 and y_2 by 3), the discrete-time analogue of the declared time rescaling of Section 3. The optimiser, restart discipline and epoch budget are identical to Section 3.3, so differences between the formulations are the formulation’s. The sweep is $q \in \{2, 4, 8, 16, 32\}$ times three seeds, fifteen runs.

Because the trained map will be judged against the scheme it imitates, the scheme itself is verified first, independently of any network, in three steps:

1. the tableau is constructed for each q from the Legendre roots and checked against every identity it must satisfy (row sums, quadrature exactness to degree $2q - 1$, the collocation conditions, and a symplecticity identity that the construction never uses, which makes it an independent check); the worst violation up to $q = 32$ is 2×10^{-16} , machine precision;
2. an exact solver (a root-finder on the $2q$ stage equations, convergence judged on the residual itself) then provides the scheme with no network in it, and its measured convergence order on problems with known solutions matches or exceeds the theoretical $2q$; and
3. a single step of 0.8 from 100 sampled states converged in 100 of 100 cases at every q , establishing that the step is well-posed on the whole training region.

Evaluation is on a held-out 21×21 uniform grid over the rectangle (441 states, disjoint from the training samples by construction, with the origin at its centre) and, for chained evaluation, on a held-out test set of 108 starting states; all ground truths are regenerated by the reference integrator, stopping exactly at each stage time.

4.5 Results: the error floor

Training gives no trouble: all fifteen runs converge to losses of 7×10^{-6} to 2.6×10^{-5} with no collapse and no divergence, and, unlike every failed run of Sections 3.4 and 3.5, the loss is an honest proxy for the error: a squared-residual loss of 10^{-5} resolves outputs to roughly its square root, $3\text{--}5 \times 10^{-3}$, which is where the held-out error lands.

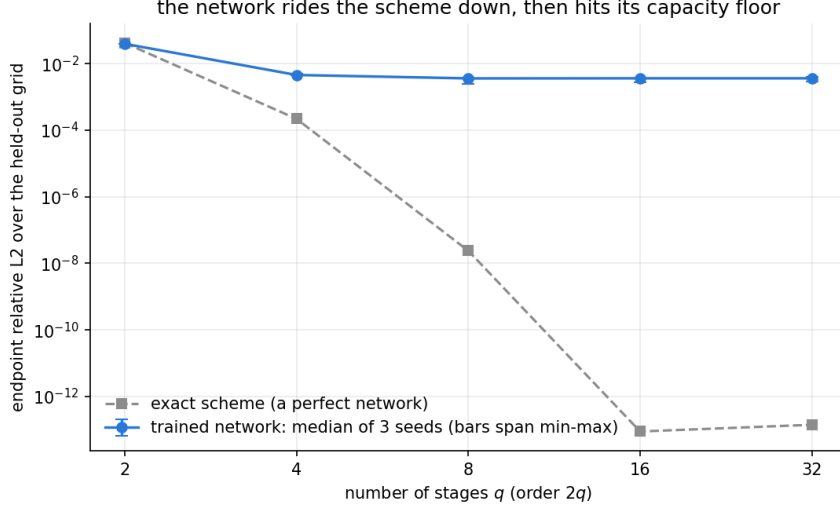


Figure 18: Endpoint relative L_2 error versus number of stages: the trained network (median of three seeds, min–max whiskers) rides the scheme’s error down and then flattens onto its floor, while the exactly solved scheme (grey) continues to machine precision.

Table 8: The capped measure of Eq. (2) on the 441-state grid (median over three seeds of the per-run values; exactly one capped point per run, the state whose true image is the origin). The mean column is pinned at that point’s fixed $1/441 \approx 2.3 \times 10^{-3}$ contribution from $q \geq 4$ and resolves nothing below it; this is the reporting rule of Section 2.3 in action.

q	median ρ	mean ρ (dominated by the capped point)
2	7.2×10^{-5}	3.5×10^{-3}
4	1.0×10^{-5}	2.3×10^{-3}
8	6.5×10^{-6}	2.3×10^{-3}
16	6.0×10^{-6}	2.3×10^{-3}
32	5.1×10^{-6}	2.3×10^{-3}

Table 7 and Figure 18 give the central result of this section. At $q = 2$ the trained network sits *on* the exactly solved scheme’s own error: the map is scheme-limited, and even a perfect fit would be that wrong. From $q = 4$ the curves part: the scheme drops ten orders of magnitude to roundoff while the network stays flat at $3.6\text{--}4.6 \times 10^{-3}$ (within a factor 1.27 for all $q \geq 4$, all seeds agreeing). The remaining error belongs to the network rather than the scheme, so more stages cannot help; the lever is the network itself, and Section 4.7 shows that the binding constraint there is the optimisation rather than capacity. The stages are learned as accurately as the endpoint at every q (asking for 66 outputs at $q = 32$ instead of 18 at $q = 8$ degrades nothing), the largest errors sit at the corners of the training region where the dynamics are fastest, and the three seeds agree on this pattern (observed in the saved per-state error maps; not shown). The floor is a property of the method rather than an initialisation accident. All later discrete-time studies fix $q = 8$: from $q = 4$ the network sets the error, and at $q = 8$ the scheme’s own error is five orders of magnitude below the network’s, so nothing about the network is masked by the scheme.

4.6 The capped measure, and the error as a distribution

In the capped measure (Table 8) the median column tracks the floor: $\sqrt{6.5 \times 10^{-6}} \approx 2.5 \times 10^{-3}$ at $q = 8$, consistent with the relative L_2 floor as it must be. On the test set (108 held-out starting states, each chained 50 steps, six periods, by the three $q = 8$ networks of Section 4.5, 16,200 (state, time, seed) values) not a single value reaches the cap: the pooled median is 1.8×10^{-4}

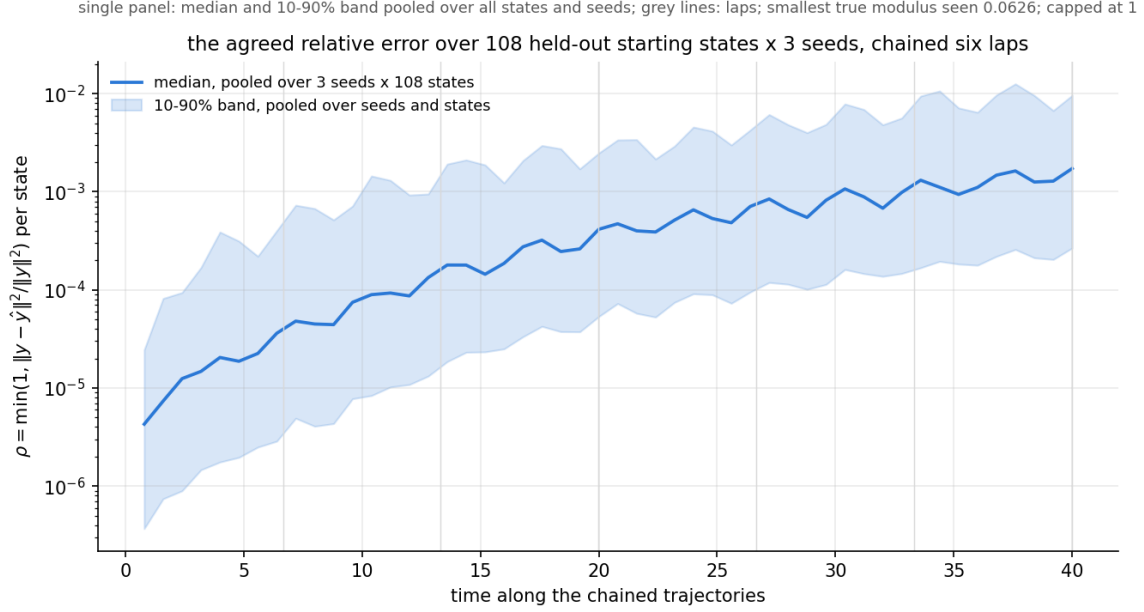


Figure 19: The error as a distribution rather than a single trajectory: the capped measure ρ over the pooled test set (three seeds $\times$ 108 held-out starting states, each chained 50 steps) against time; median and 10–90% band. The drift is slow and repeats with the oscillator’s period; nothing is excluded.

at two periods and 1.7×10^{-3} at six, the worst single value anywhere in the set is 6.6×10^{-2} , and the smallest true state modulus encountered is 0.063, so the measure is defined everywhere with no exclusions (Figure 19).

4.7 Network size, the seed, and selection

The continuous-time route’s size study has a discrete-time twin, with a different question to test: is the 3.6×10^{-3} floor a capacity limit? The design crosses depth $\{2, \dots, 10\}$ with width $\{16, 32, 64, 128, 256\}$ (45 architectures) at fixed $q = 8$, three seeds each, 135 runs, everything else frozen; each network is scored one-step on the 441-state grid and chained fifty steps (six periods) over the 108-start test set. Seven architectures spanning the usable band and its edges (2×64 , 4×32 , 4×50 , 4×64 , 4×128 , 4×256 , 8×128) were then trained at ten seeds each. For honesty the 108 test starts are split: 50 of the Latin-hypercube starts form a validation set used only to rank trained networks, and the remaining 58 (the 8 named reference starts plus 50 Latin-hypercube starts) are held out and used only to report the chosen network, so the reported numbers cannot be flattered by the selection. A network’s six-period error ranks almost identically on the two halves (rank correlation +0.989), which is what makes selection meaningful. Four findings.

The floor does not move. The best of 45 architectures (3 layers $\times$ 128 units) reaches 2.5×10^{-3} , a factor 1.5 below the 4×50 configuration’s 3.6×10^{-3} , and still five orders of magnitude above the exact scheme (Figure 20). Depth beyond about 5 hurts sharply (depths 2–5: median 4.6×10^{-3} ; depths 7–10: median 1.8×10^{-2}); width stops helping beyond 64–128; the worst corner (10×16) sits at 1.3×10^{-1} .

The binding constraint is the optimiser, not capacity. Across the 135-run grid, the Spearman rank correlation between converged training loss and held-out one-step error (the correlation of the two quantities’ rank orderings, so that +1 means the run with the k -th smallest loss also has the k -th smallest error for every k) is +0.974: architectures that fail also fail to train. There is no capacity-rich, badly-generalising regime, and every run used its full budget of six restarts. The floor of Section 4.5 is an optimisation floor. (The same training loss ranks six-period chained error at only +0.35; the distinction between the two errors matters and is the subject of the

next two findings.)

Long horizons are decided by the seed, not the architecture. I measure error growth under chaining as the ratio of a network’s median six-period ρ to its median one-period ρ . This ratio varies $\sim 13\times$ between seeds of a single architecture and $\sim 300\times$ across the grid; the 4×50 configuration’s ten-seed median ratio is 28, with only one seed in ten staying within a factor 3 of its one-period error. For no architecture does the error reliably stop growing. Comparing each candidate’s ten-seed six-period errors against the 4×50 configuration with Mann–Whitney tests, Bonferroni-corrected across the candidates (threshold $p < 0.0083$), the width-64 architectures’ absolute improvement is significant ($p \approx 0.006$) while the three-seed grid’s apparent best, 4×128 , is not ($p = 0.12$); and even the significant architectures’ seed spreads span decades (Figure 21).

But growth is a measurable, transferable property of the trained network. A validation chain to the target horizon predicts held-out six-period error at Spearman $+0.98$; the training loss and the one-step error predict it at only $+0.35$, and a one-period chain at $+0.67$. Hence the selection protocol:

1. train about ten seeds of the chosen architecture;
2. chain each on the validation starts to the target horizon; and
3. keep the seed with the smallest validation error.

Selection beats the median seed by $3\text{--}96\times$ per architecture. Ranked by the six-period validation chain and reported on the held-out half, the selected seeds of the seven candidates reach six-period ρ between 2.0×10^{-6} (4×32) and 4.8×10^{-5} (8×128); the 4×32 candidate gave the smallest held-out six-period error. A far-horizon check then separated the front-runners: chained to thirty periods ($t = 200$) from twenty fresh starts, the 4×32 choice moved from 3.1×10^{-6} to 3.4×10^{-6} (a factor 1.1), while the 4×64 , 4×128 and 4×50 choices grew by factors 11.3, 8.7 and 24 (Figure 22). On this basis the 4×32 seed-9 network (3,858 parameters) is the network reported throughout the rest of the paper. The far-horizon check also carries a caveat: a six-period validation window does not guarantee thirty periods for every candidate, so the protocol’s window should match the horizon the map will be used at.

4.8 Whole trajectories: chaining versus one large step

How should the one-step technique deliver the actual goal, a starting state in and a whole trajectory out? The source work’s discrete-time formulation can be read in two ways. Route A chains the trained one-step map: feed the endpoint back in, no retraining, reaching a horizon T in steps of $\Delta t = 0.8$ (50 chained steps at $T = 40$, six periods). Route B generalises the source work’s demonstration on the Allen–Cahn equation, which it solves with a single large implicit step: here, one network learns one large step whose stage states are the trajectory readout. Route B’s feasibility was checked first with the exact solver: at a full period in one step ($\Delta t = 6.66$) the stage equations stop being reliably solvable even at $q = 64$ (solver success 90 of 100 states). A-stability is a linear property; what actually binds is nonlinear solvability. The largest feasible large step is half a period ($\Delta t = 3.2$ at $q = 64$), and Route B was trained there (three seeds). The classical comparison of Section 3.4 sits on the other side of the same trade: at $\Delta t = 0.8$ the explicit order-6 method is not finite past $t = 8.8$, while the implicit scheme the network imitates takes that step at 2.4×10^{-8} one-step error, and the trained map at 3.6×10^{-3} .

Table 9 and Figures 23 and 24 give the result. The 4×50 seeds compound gently, from 2.6×10^{-3} at half a period to 2.6×10^{-2} at six; the selected network’s column is flat at $1.6\text{--}2.1\times 10^{-3}$ from half a period to six periods (the $T = 40$ value over the $T = 3$ value is a factor 1.3). The map has stopped compounding, and at six periods it is twelve times better than the best unselected seed. Against Route B at the matched horizon ($T = 3.2$, i.e. four chained steps

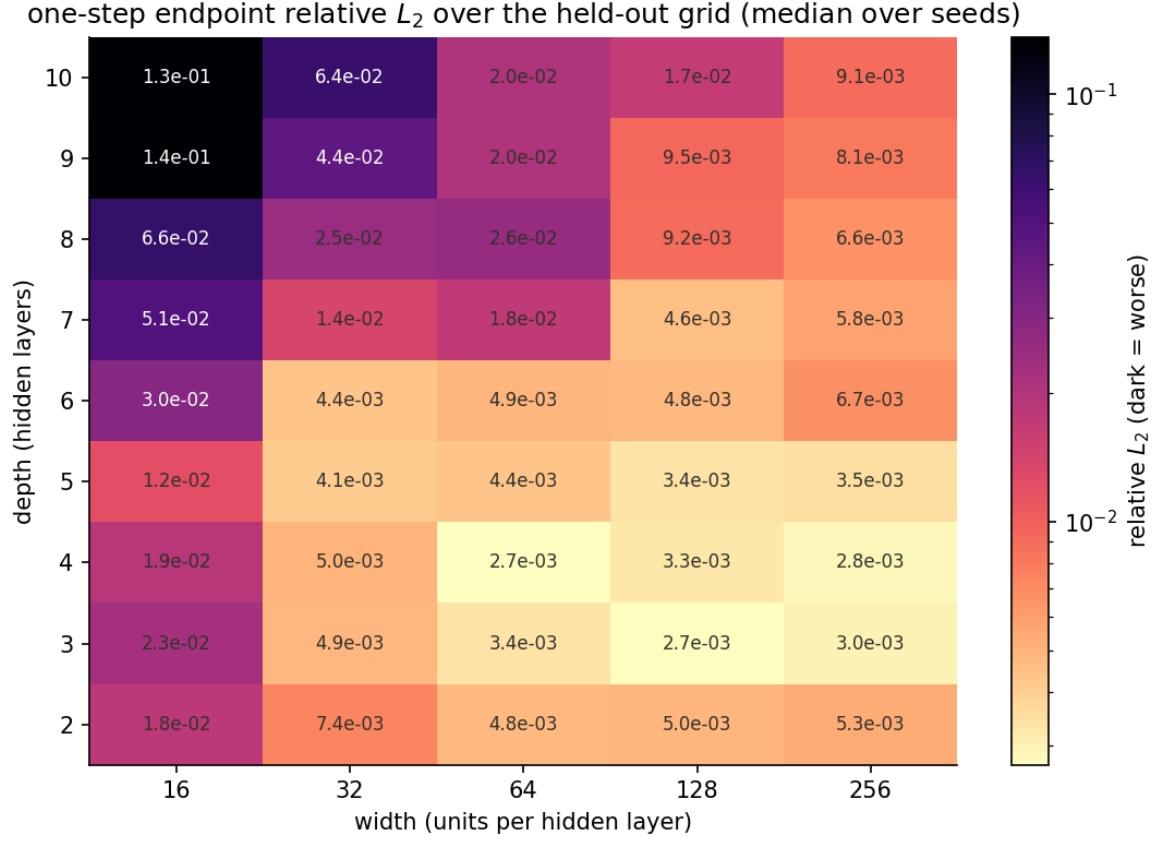


Figure 20: One-step error over the architecture grid at fixed $q = 8$: no entry approaches the exactly solved scheme; depth harms, width stops helping beyond 64–128. The floor is not a capacity floor.

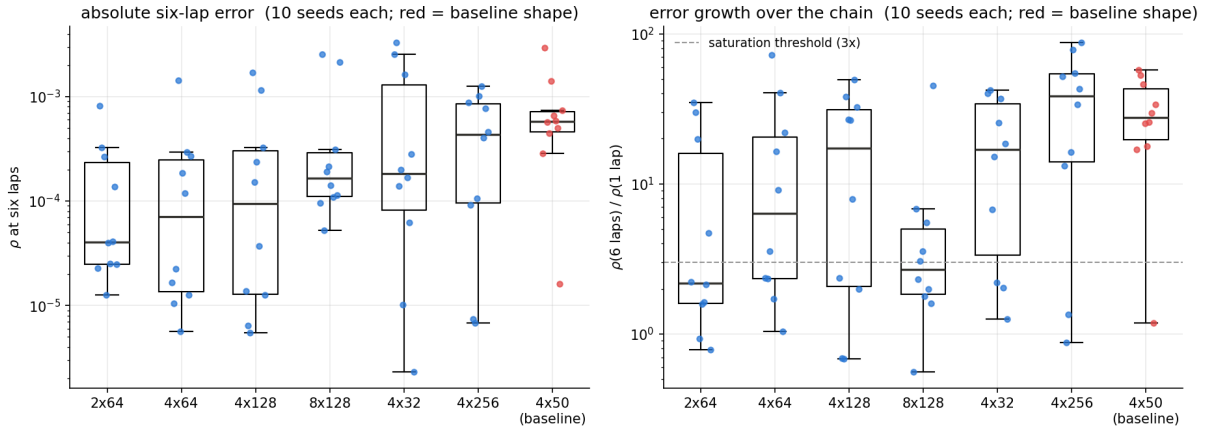


Figure 21: Ten seeds for seven architectures, six-period chained error: the seed-to-seed spread within one architecture rivals the whole grid’s spread. The seed, not the shape, decides the horizon.

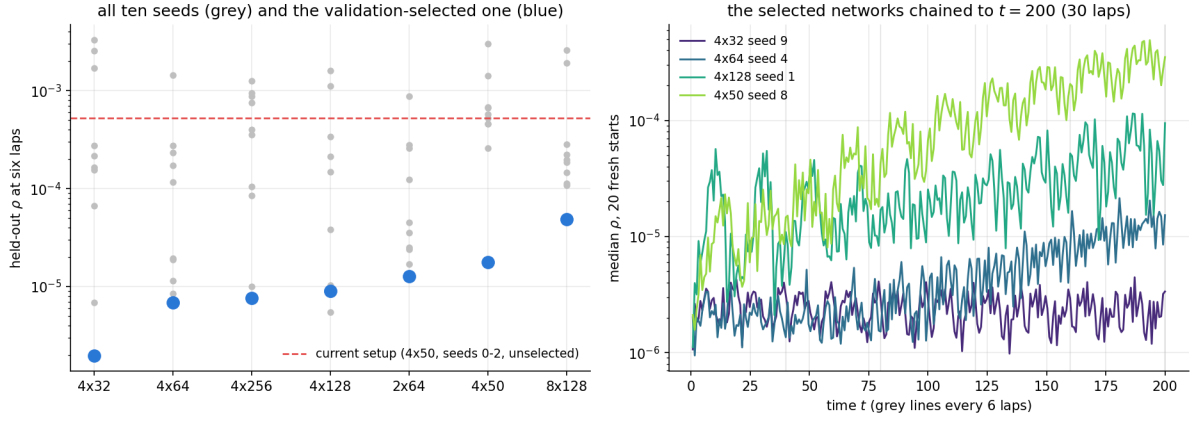


Figure 22: The selection payoff and the far-horizon check: validation-selected networks against their unselected siblings, and the selected 4×32 network holding $\rho \approx 3 \times 10^{-6}$ flat to thirty periods on fresh starts.

Table 9: Whole trajectories over the continuous-time sweep’s horizons: relative L_2 error over all chained step times up to T , per start, with the median pooled over the three seeds and the 108 test starts (single network for the selected column). The last column lists, for scale, the plain variant’s dedicated single-start fits of Section 3.4; the causally weighted variant reaches 7.6×10^{-5} at $T = 7$ and 2.1×10^{-4} at $T = 14$ for its single training start (Section 3.5) and exceeds unity at $T \geq 27$.

T	Route A, 4×50 seeds	Route A, selected 4×32	continuous-time, plain
3	2.6×10^{-3}	1.6×10^{-3}	3.8×10^{-5} (dedicated fit, one start)
7	4.8×10^{-3}	2.1×10^{-3}	6.6×10^{-4} (dedicated fit, one start)
14	9.0×10^{-3}	2.1×10^{-3}	0.79
27	1.8×10^{-2}	2.1×10^{-3}	0.90
40	2.6×10^{-2}	2.0×10^{-3}	0.94

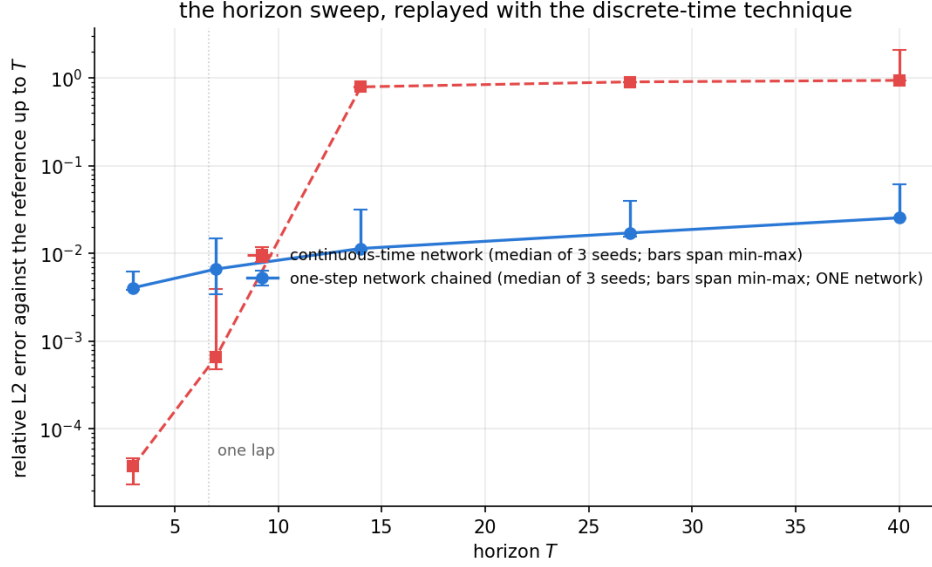


Figure 23: Both formulations on one axis, at the continuous-time sweep’s horizons; the continuous-time points show the plain variant (the causally weighted variant reaches 2.1×10^{-4} at two periods for its single training start and exceeds unity from four). The chained map’s pooled median rises gently where the continuous-time fits fail at order one; below one period the dedicated fits are the more accurate.

of 0.8 against one large step of 3.2): Route A selected 2.1×10^{-3} versus Route B 2.0×10^{-2} , so chaining beats the one-step-per-horizon mode by a factor 10 (already $6.6 \times$ with the 4×50 seeds, 3.0×10^{-3} versus 2.0×10^{-2} ; Figure 25). Chained trajectories may briefly leave the training rectangle (4 starts of 108, per seed); all return within one step. The loop’s attraction corrects the radial direction on its own, and the surviving error is mostly slow drift in where along the loop the state sits at a given time, visible as the settled period: 6.6573 measured against the true 6.663287 (-0.09%).

Two caveats apply to the comparison with the continuous-time column. The continuous-time error is evaluated on the dense reference grid while the chained error is evaluated at the step times, and each continuous-time network was trained for its specific horizon and single starting state while a single discrete map serves all horizons and all 108 starts: the comparison is between formulations rather than matched estimators. The other direction is also visible in Table 9: below one period the dedicated continuous-time fits are more accurate (up to forty times, for the plain fits at $T = 3$), and the causally weighted fit at $T = 14$ beats the chained map’s pooled median for its one trained start. The chained map’s advantage is that it serves every start in the region at every horizon; from four periods onward no continuous-time variant produces a usable trajectory while the chained map’s pooled median stays at 2×10^{-3} .

4.9 Training from labels instead: a matched comparison

If perfect solution data were available, would it beat the physics? The test is a matched pair: identical network, identical seeded initial weights, identical optimiser and budget; the only differences are the loss, *physics* (the reconstructions of Eq. (7), as everywhere above) versus *data* (mean squared error to the true stage states and endpoint, computed by the reference integrator), and the training-set size $n \in \{50, 125, 250, 500, 1000, 2000\}$, three seeds each, 36 runs.

Both variants flatten from $n \approx 500$: the physics medians read $3.4, 3.4, 3.6 \times 10^{-3}$ at $n = 500, 1000, 2000$ and the data medians $1.6, 1.7, 1.6 \times 10^{-3}$, a stable factor of about 2.1 apart, with the physics variant ahead only at $n = 50$ (Table 10 and Figure 26; the $n = 2000$ physics entry

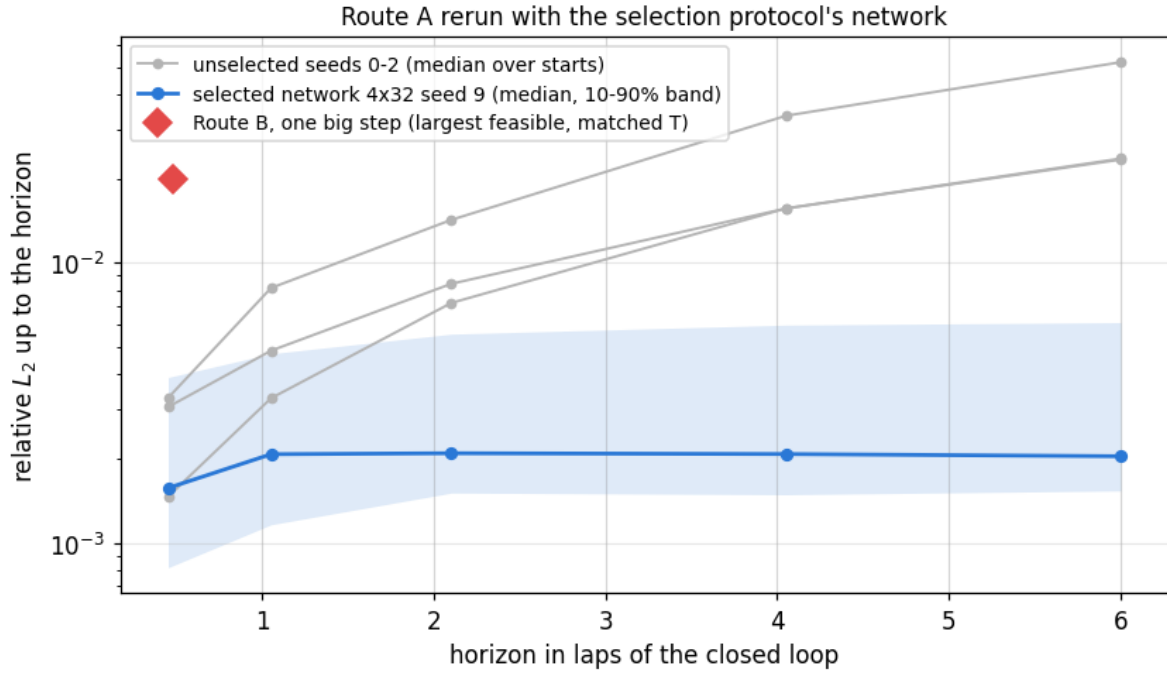


Figure 24: The selected network against the unselected 4×50 seeds, chained over the test set: a compounding $2.3\text{--}5.2 \times 10^{-2}$ six-period map becomes a flat 2.0×10^{-3} one.

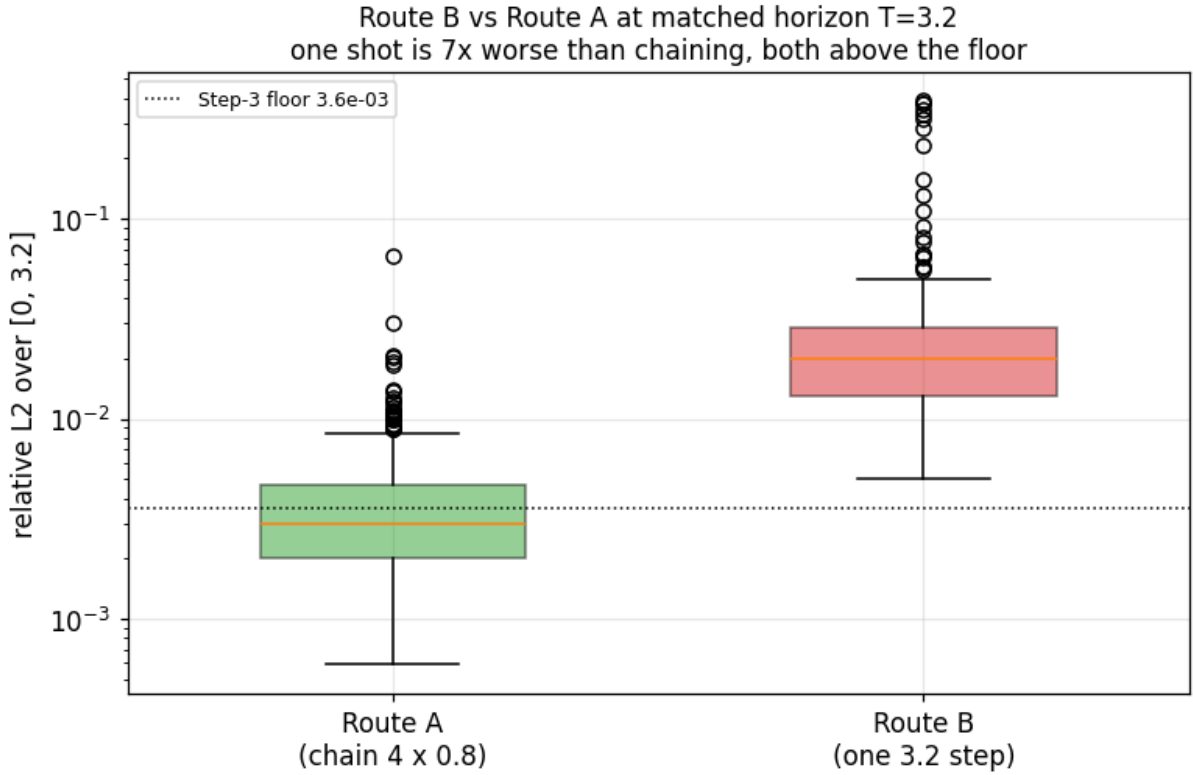


Figure 25: Route B (one large half-period step, stage states as the trajectory readout) against Route A at the matched horizon: the one-step-per-horizon mode is an order of magnitude worse, and a full-period step is not even reliably solvable.

Table 10: Physics-trained versus data-trained (perfect labels) at matched everything else: endpoint relative L_2 on the 441-state grid, medians over three seeds, against training-set size n . The last column is the reference-integrator work needed to label the data variant’s training set, at 799 steps per state.

n	physics (median)	data (median)	integrator steps for labels
50	9.2×10^{-3}	9.8×10^{-3}	39,950
125	6.3×10^{-3}	3.0×10^{-3}	99,875
250	4.1×10^{-3}	1.8×10^{-3}	199,750
500	3.4×10^{-3}	1.6×10^{-3}	399,500
1000	3.4×10^{-3}	1.7×10^{-3}	799,000
2000	3.6×10^{-3}	1.6×10^{-3}	1,598,000

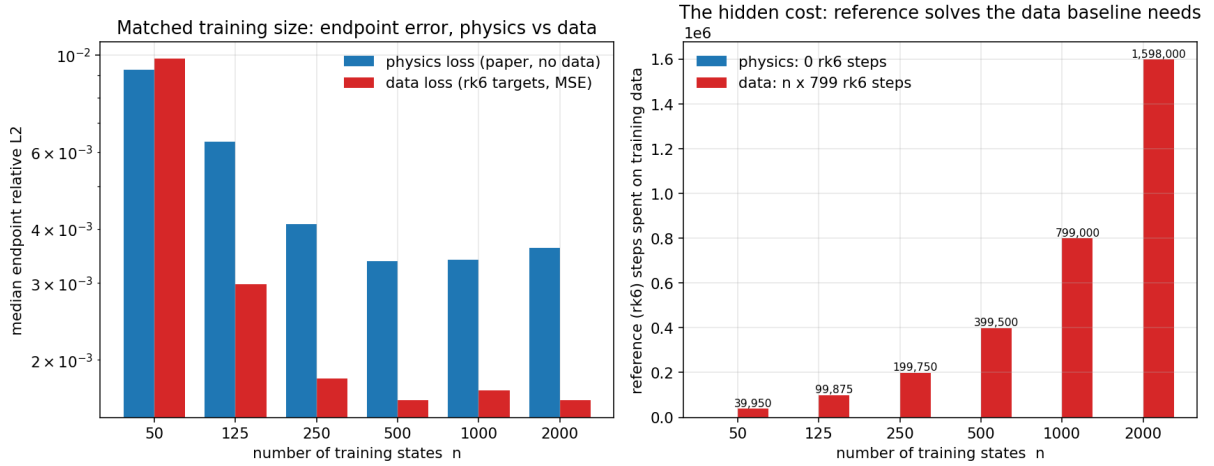


Figure 26: Accuracy against training-set size for the matched pair (left) and the reference-integrator work behind the data variant’s labels (right): the physics variant’s reference-data bill is zero everywhere.

is the run of Section 4.5). The last column is the important one, and its accounting is explicit: the labels for one training state are the eight true stage states plus the true endpoint, produced by running the reference integrator of Section 2 across the step at its working step size 10^{-3} , halting exactly at each stage time; that is 799 integrator steps of 10^{-3} to traverse the step of 0.8 once, so the column reads $799n$.⁴ The physics variant’s reference cost is exactly zero at every n . The physics loss therefore gives up a factor of about 2.1 in accuracy relative to perfect labels and in exchange needs no reference data at all. On the source of the remaining gap: the data loss’s minimiser is the true stage states, while the physics loss’s minimiser is the scheme’s stage states; at $q = 8$ these differ by about 2.4×10^{-8} on this grid (Table 7), five orders below either variant’s error, so at this accuracy the gap between the variants is not an information gap, and the findings are consistent with the two losses differing in the path the optimiser takes rather than in what they specify.

5 Discussion

Relation to the source work. Where the regimes overlap, the reproduction agrees: on sub-period horizons, the same regime as the source work’s continuous-time solution demonstrations (the Burgers equation on $t \in [0, 1]$; the Schrödinger equation on $t \in [0, \pi/2]$), the transcription meets or betters the reported accuracy of a few 10^{-4} to 2×10^{-3} . Where the source work stops, the measurements sharpen it in three ways. First, it frames the continuous-time formulation’s

⁴Outside synthetic tests, labels of this quality are not available; even these carry the full reference method inside them.

principal limitation as the *cost* of collocation in higher dimensions. On this problem the collocation budget is trivial and fully paid (Section 3.2), was then varied 64-fold and re-placed with no effect (Sections 3.7 and 3.8), and the formulation fails anyway, through optimisation collapse onto zero-residual spurious solutions with a converged loss. This runs directly against its stated expectation, and across 733 runs rather than the 15 of the reproduction alone, so cost is not the binding limitation on long horizons even in one dimension. Second, on the discrete-time side the source work attributes its Allen–Cahn error to network capacity, and its appendix tables already contain a visible error floor (its q -sweep at $\Delta t = 0.8$ flattens at 7×10^{-4} – 1.2×10^{-3} from $q = 16$ to $q = 500$); what it lacks is the exactly solved scheme on the same states and metric, which is what turns a plateau in a table into an attribution (scheme-limited at $q = 2$, network-limited from $q = 4$). The 135-run architecture sweep then goes one step further than the source work’s capacity attribution: the +0.974 rank correlation between training loss and one-step error makes the floor an optimisation floor, which no network size fixes. Third, its stage-count prescription (choose q so that the theoretical $\mathcal{O}(\Delta t^{2q})$ truncation reaches machine precision, giving $q \approx 81$ at $\Delta t = 0.8$) is far past the point of usefulness once the optimisation binds: at $q = 8$ the scheme already sits five orders of magnitude below the floor. Finally, the source work’s suggestion that a single large implicit step can resolve the entire solution does not transfer unconditionally: on this system the nonlinear stage equations stop being reliably solvable near a full-period step, a solvability limit, distinct from stability, that the source work does not discuss, and at the largest feasible step chaining beats the single-step mode by a factor 10 (Section 4.8).

Relation to the causal-weighting literature. Causal weighting behaves as its authors report [4]: it addresses the pathology it targets, the simultaneous fitting of all times, at a training cost that grows quickly with the domain, and their own guidance that causal weighting complements rather than replaces time-marching [7] is precisely what Sections 3.5 and 4.8 measure from the two sides. A fixed look-back window (weighting each point only by the residual of the preceding period) was considered and rejected without an experiment: a fit that jumps once and then follows a different genuine solution of the equation has zero residual inside every window older than the jump, so windowing re-admits exactly the silent failures of Section 3.4. The whole-history integral in Eq. (4) is what keeps the not-yet-reached part of the domain out of the fit, and the same integral is what makes the weight condition harder to meet as the horizon grows. The density study adds one fact to this literature: the weight condition itself has a density floor (about 20 points per unit time here) below which it is met on wrong solutions (Section 3.7). The failure modes measured here sit within the broader literature on PINN optimisation pathologies [8, 9], with more instrumentation per failure than is usual.

The recurring observation. The equation alone admits many solutions, and any stretch of the domain the loss cannot see is eventually exploited by the optimiser: late times in the plain formulation, the intervals between collocation points in the causally weighted one. Each modification closed one such opening and exposed the next, at increasing cost. The resource axes are now all measured, and none changes the outcome, so what remains is the starting state’s position. Two corollaries deserve emphasis. Phase-plane agreement is necessary but never sufficient: an orbit can lie exactly on the limit cycle and be order-one wrong in time (Figure 10). And in every failure the training loss was small: the verdict must always come from the error against a trusted reference at points the training never saw, and never from the loss. The discrete-time formulation removes the disease rather than treating it: every output is tied to a known state through the algebra of the step, so a spurious solution has nowhere to live. Consistently with that, it is also the only formulation here whose loss predicts its one-step error, all the way to a rank correlation of +0.974 across 135 architectures.

6 Future work

Five studies follow naturally from these measurements; I list them in the order I would run them.

- **A sweep of the step size.** Every discrete-time result in this paper is at the single step $\Delta t = 0.8$ taken from the source work. The trade between step size, stage count, solvability and network error is unmeasured, and the chained map’s error budget per period depends directly on it; this is the axis I would test first.
- **Attribution of the optimisation floor.** The 135-run sweep rules out capacity; what it does not yet separate is optimiser dynamics from the loss landscape: whether a different optimiser class, training schedule, or curriculum over the training region moves the 3.6×10^{-3} floor at fixed architecture.
- **The ε schedule.** The measured hardening of the weight condition with horizon suggests the variant $\varepsilon \propto 1/T$, which directly compensates the whole-history integral’s growth; the placement and density studies fix everything needed to test it in isolation.
- **Inverse problems.** The source work’s data-driven discovery setting, recovering the damping parameter μ from noisy observations, has a natural discrete-time variant that reuses exactly the trained reconstructions with μ as a free parameter.
- **Learned quadrature.** A variant in which the network predicts the step’s weights rather than its states, for which the fine-grained single-step reference built for this study provides the supervision.

7 Conclusion

On a fully controlled nonlinear oscillator with a reference solution trusted to 10^{-13} , the continuous-time PINN formulation is excellent below one period of the limit cycle and fails structurally beyond it, collapsing onto zero-residual spurious solutions while its training loss converges. In these measurements, no resource changed that outcome: ample collocation, a four-fold optimisation budget, 25 network architectures at 600 runs, a 64-fold density range, and targeted re-placement of the collocation points. The strongest loss-level modification moves the failure out by one period at a cost that grows faster than linearly, and what orders the outcomes is the starting state: success at two periods occurred in 19 of 150 causally weighted runs at the 1% threshold, and at four periods in none of 300. The discrete-time formulation, generalised into a reusable one-step map over starting states, inherited none of this in these tests. Its supervision is anchored to a known state by the algebra of an implicit Runge–Kutta step, its training loss predicts its one-step error, and its accuracy is cleanly attributable: scheme-limited at two stages, floored at 3.6×10^{-3} from four by the optimisation rather than by capacity. Its long-horizon quality, decided by the training seed, is measurable on a validation chain and controlled by a simple selection protocol. The selected network, chained without retraining, carried 108 held-out starting states through six periods at 2.0×10^{-3} relative error, held flat to thirty periods on fresh starts, and beat the single-large-step reading of the source work by a factor 10 at the largest step the nonlinear algebra allowed; a matched data-trained twin shows the physics loss costing a factor of about 2.1 in accuracy against perfect labels, with no reference data needed in exchange. For initial-value problems on long horizons, these findings favour formulations that re-anchor the physics on known states step by step; and on this problem the binding failure of the continuous-time formulation was not the cost of enforcing the equation, but the optimiser’s freedom to satisfy it on the wrong solution.

Data availability

All training scripts, saved results, and the notebooks that generate every figure and number in this paper are available at https://github.com/GeorgeWilliam1999/Van_Der_Pole. All experiments are seeded and reproduce deterministically; the reference solution and its verification are included.

References

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- [2] B. van der Pol. On “relaxation-oscillations”. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.
- [3] R. Aaij et al. Allen: A high-level trigger on GPUs for LHCb. *Computing and Software for Big Science*, 4:7, 2020. doi: 10.1007/s41781-020-00039-7.
- [4] Sifan Wang, Shyam Sankaran, and Paris Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421: 116813, 2024. doi: 10.1016/j.cma.2024.116813. arXiv:2203.07404.
- [5] Álvaro Fernández Corral, Nicolás Mendoza, Armin Iske, Andrey Yachmenev, and Jochen Küpper. Learning phase-space flows using time-discrete implicit Runge–Kutta PINNs, 2024. Scientific Computing and Machine Learning (SCML) 2024.
- [6] Ernst Hairer and Gerhard Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*. Springer, Berlin, Heidelberg, 2 edition, 1996. doi: 10.1007/978-3-642-05221-7.
- [7] Sifan Wang, Shyam Sankaran, Hanwen Wang, and Paris Perdikaris. An expert’s guide to training physics-informed neural networks, 2023.
- [8] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021. doi: 10.1137/20M1318043.
- [9] Aditi S. Krishnapriyan, Amir Gholami, Shandian Zhe, Robert M. Kirby, and Michael W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 26548–26560, 2021.